

Trabajo Práctico 1 - Grupo 02

REPORTE

Integrantes:

- Bermudez, Agustin. Padrón: 111863
Email: abermudez@fi.uba.ar
- Calderón, Tiago. Padrón: 111894
Email: tcalderon@fi.uba.ar
- Gonzalez Pautaso, Mateo. Padrón: 111699
Email: magonzalezp@fi.uba.ar
- Moreyra, Santiago. Padrón: 110531
Email: smoreyra@fi.uba.ar
- Nieves, Maylen. Padrón: 111271
Email: mnieves@fi.uba.ar

REPORTE	1
EJ1: Análisis Exploratorio de Datos	3
1. Descripción del Dataset y Variables Relevantes	3
2. Hipótesis Iniciales y Supuestos de Trabajo	4
3. Preprocesamiento: Estudio de correlaciones y análisis de distribuciones	4
4. Tratamiento de valores nulos	8
5. Detección y Tratamiento de Valores Atípicos (Outliers)	9
6. Generación de Nuevos Features	10
7. Visualizaciones y Respuestas a Preguntas de Investigación	10
EJ2: Modelos de Clasificación Binaria	13
1. Descripción del dataset y clasificación de variables	13
2. Tratamiento de valores faltantes	13
3. Detección y Tratamiento de Valores Atípicos (Outliers)	14
4. Feature Engineering, Encoding y Normalización	14
5. Entrenamiento y evaluación: Árbol de Decisión	16
6. Entrenamiento y evaluación: Random Forest	18
7. Entrenamiento y evaluación: XGBoost	20
8. Comparación de Modelos	22
EJ3: Modelos de Regresión	23
1. Breve descripción y análisis del dataset	23
2. Preparación de la variable target	23
3. Filtrado de columnas irrelevantes	24
4. Clasificación de variables	27
5. Filtrado por reglas de negocio	28
6. Análisis exploratorio	28
7. División train/test	35
8. Feature Engineering	36
9. Tratamiento de nulos y escalado	36
10. Entrenamiento y predicción	37
11. Elección del modelo	43
EJ4: Clustering	44
1. Breve descripción y análisis del dataset	44
2. Escalado	45
3. Tendencia al clustering	45
4. Estimación del número de grupos	45
5. Visualización de los grupos	49
6. Descripción de los grupos	49
Tiempo dedicado	50

EJ1: Análisis Exploratorio de Datos

1. Descripción del Dataset y Variables Relevantes

El análisis se centró en los viajes realizados en taxis Yellow Cab de la ciudad de Nueva York durante el primer trimestre del año 2025. Descargamos directamente los archivos en formato .parquet de la página de la [TLC](#), decidimos unir toda la información en un mismo Dataframe concatenando por filas. Debido a esto, la cantidad de filas iniciales fue de 11.198.026 y la cantidad de columnas fue de 20.

Variable	Tipo estadístico	Tipo en pandas y su corrección
VendorID	Cualitativa nominal	int64 → category
tpep_pickup_datetime	Temporal	datetime64
tpep_dropoff_datetime	Temporal	datetime64
passenger_count	Cuantitativa discreta	float64 → Int64
trip_distance	Cuantitativa continua	float64
RatecodeID	Cualitativa nominal	float64 → category
store_and_fwd_flag	Cualitativa nominal	object → category
PULocationID	Cualitativa nominal	int64 → category
DOLocationID	Cualitativa nominal	int64 → category
payment_type	Cualitativa nominal	int64 → category
fare_amount	Cuantitativa continua	float64
extra	Cuantitativa continua	float64
mta_tax	Cuantitativa continua	float64
tip_amount	Cuantitativa continua	float64
tolls_amount	Cuantitativa continua	float64
improvement_surcharge	Cuantitativa continua	float64
total_amount	Cuantitativa continua	float64
congestion_surcharge	Cuantitativa continua	float64
Airport_fee	Cuantitativa continua	float64
cbd_congestion_fee	Cuantitativa continua	float64

Las variables más relevantes según nuestro análisis son:

- **tpep_pickup_datetime / tpep_dropoff_datetime**: marca temporal de inicio y fin del viaje, base del análisis temporal.
- **trip_distance** (continua): distancia recorrida en millas.
- **fare_amount / total_amount** (continuas): tarifa base y monto total incluyendo cargos y propina.

- **tip_amount** (continua): propina registrada exclusivamente para pagos con tarjeta.
- **PULocationID / DOLocationID** (nominales): zona TLC de inicio y fin, enriquecidas mediante merge con el archivo `taxi_zone_lookup.csv` (borough, zone, service_zone).
- **RatecodeID** (nominal): tipo de tarifa (estándar, JFK, Newark, etc.).
- **payment_type** (nominal): método de pago (tarjeta, efectivo, desconocido, etc.).
- **passenger_count** (discreta): cantidad de pasajeros (rango legal 1-6).

Para enriquecer el componente geográfico, se integró el archivo de zonas de la TLC (`taxi_zone_lookup.csv`), mapeando los ID con sus respectivos distritos (boroughs) y barrios. Además, se revisó la información proporcionada por el [diccionario](#) de la misma empresa.

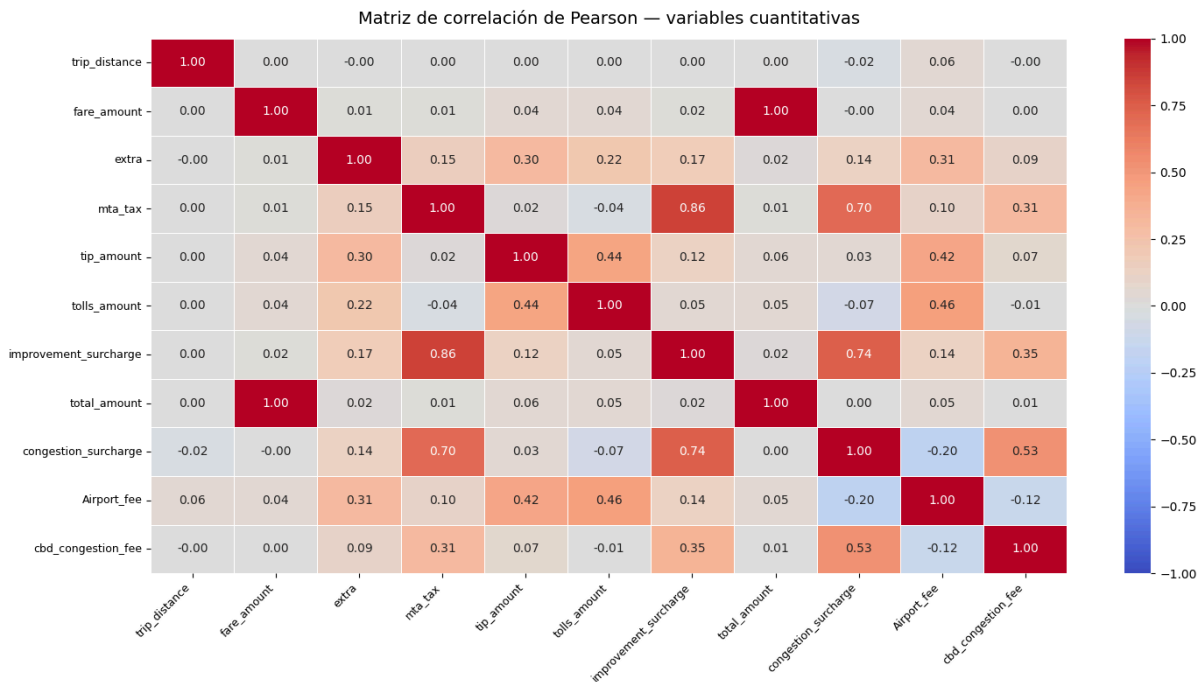
2. Hipótesis Iniciales y Supuestos de Trabajo

Como guía para el análisis descriptivo, se plantearon preguntas enfocadas en la dinámica operativa y económica del servicio:

- **¿Cómo varía la demanda de taxis a lo largo del día y la semana?** Se supuso la existencia de picos en días hábiles durante las franjas de entrada y salida laboral, contrastando con una fuerte actividad nocturna los fines de semana.
- **¿Los viajes en hora pico son más lentos aunque recorran distancias similares?** Se hipotetizó que la congestión en estos horarios impactaría directamente reduciendo la velocidad promedio.
- **¿Qué zonas y distritos generan los viajes más rentables?** La intuición apuntó a las zonas centrales de Manhattan y a los principales aeropuertos de Queens.
- **¿Existe una diferencia sustancial en el perfil económico de los viajes aeroportuarios?** Se esperaba registrar un ticket y propinas significativamente más altos.
- **¿El comportamiento de la propina depende de la distancia y la cantidad de pasajeros?** Se espera que la tasa de propina crezca con la distancia aunque no tenemos una hipótesis clara respecto a la cantidad de pasajeros.

3. Preprocesamiento: Estudio de correlaciones y análisis de distribuciones

Antes de proceder con la imputación y limpieza de los datos, se realizó un estudio de correlación de Pearson sobre las variables cuantitativas continuas para verificar variables que pueden contener la misma información.

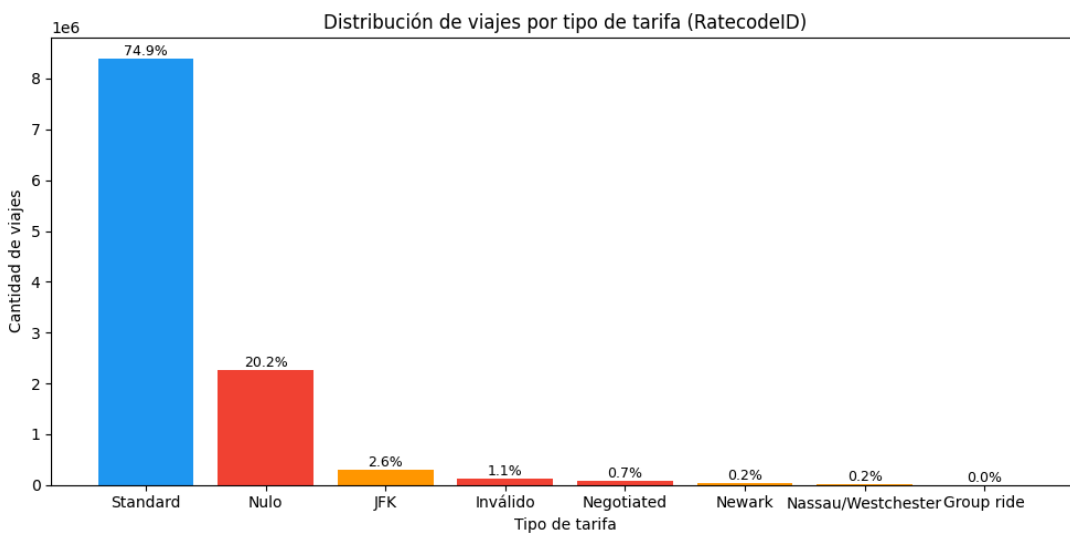
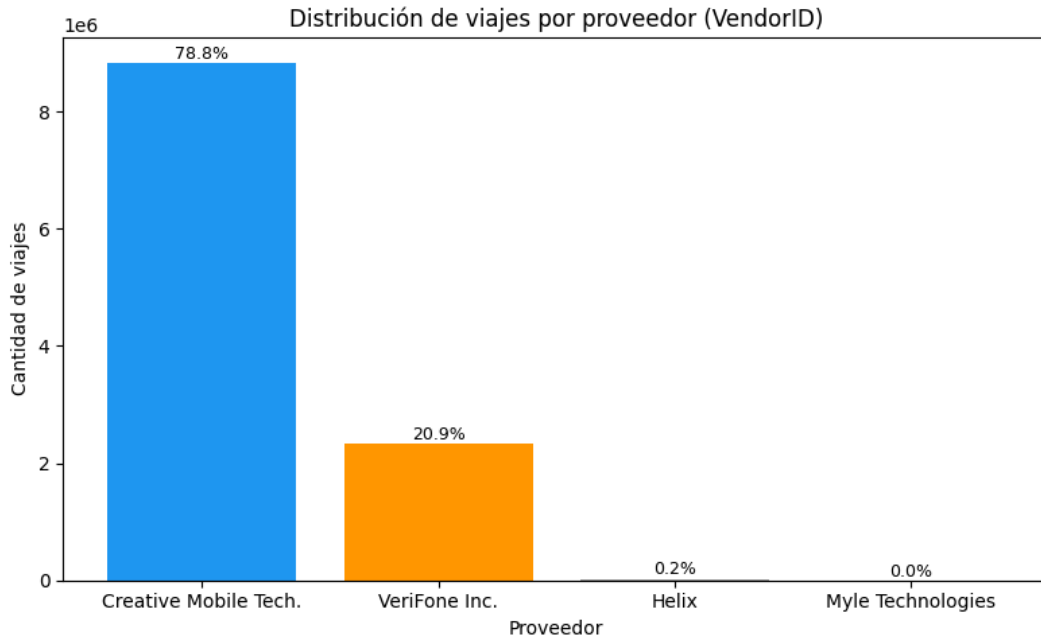


La matriz de correlación reveló correlaciones que toman sentido dentro de la lógica de negocio:

- Colinealidad perfecta (1.0) entre `fare_amount` y `total_amount`, debido a que la primera es casi igual a la segunda pero con el adicional de algunos cargos pequeños.
- Una fuerte correlación (0.86) entre costos fijos que se aplican bajo las mismas condiciones, como `mta_tax` e `improvement_surcharge`.
- También se detectaron relaciones medias lógicas, como la de `airport_fee` con `tolls_amount` (0.46) y `tip_amount` (0.42), indicando que los viajes aeroportuarios suelen incurrir en peajes y generar mayores propinas.
- Una correlación digna de mencionar es la de `trip_distance` y `fare_amount`, la cual es casi nula. Lo que es extraño ya que se supone que a mayor distancia el precio sería más elevado. Este comportamiento podría estar siendo causado por outliers dentro de los datos.

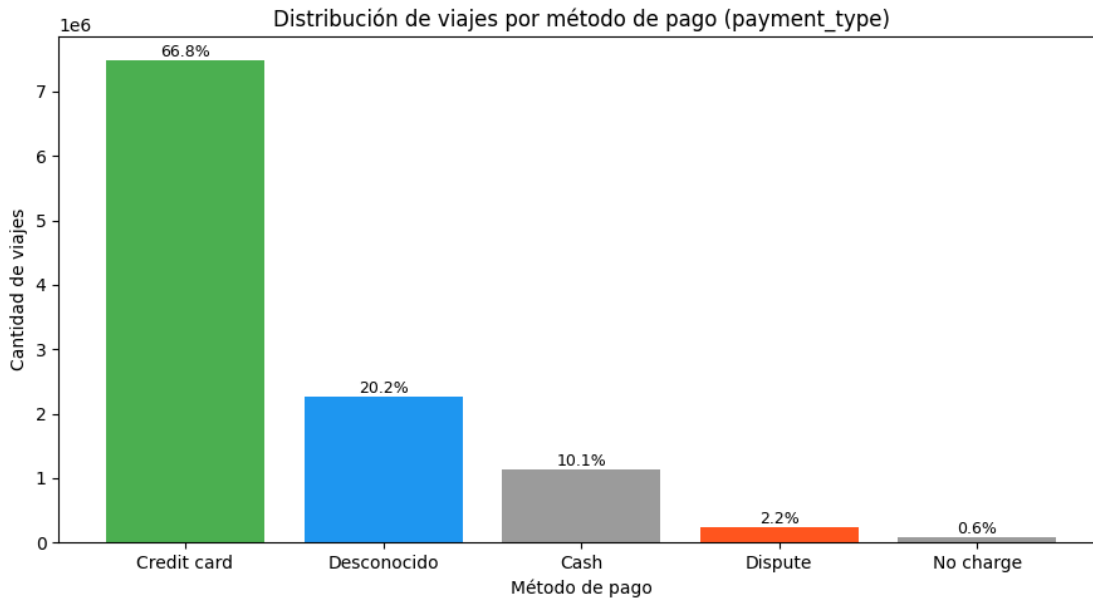
Posteriormente, se estudiaron las distribuciones categóricas para identificar patrones y prevenir sesgos:

- **Dominancia de tarifas y proveedores:** El mercado se encuentra operado casi en su totalidad por dos proveedores (Creative Mobile Tech. y VeriFone Inc.), mientras que a nivel tarifario, la tarifa estándar (`RatecodeID 1`) es la categoría más repetida con mucha diferencia.



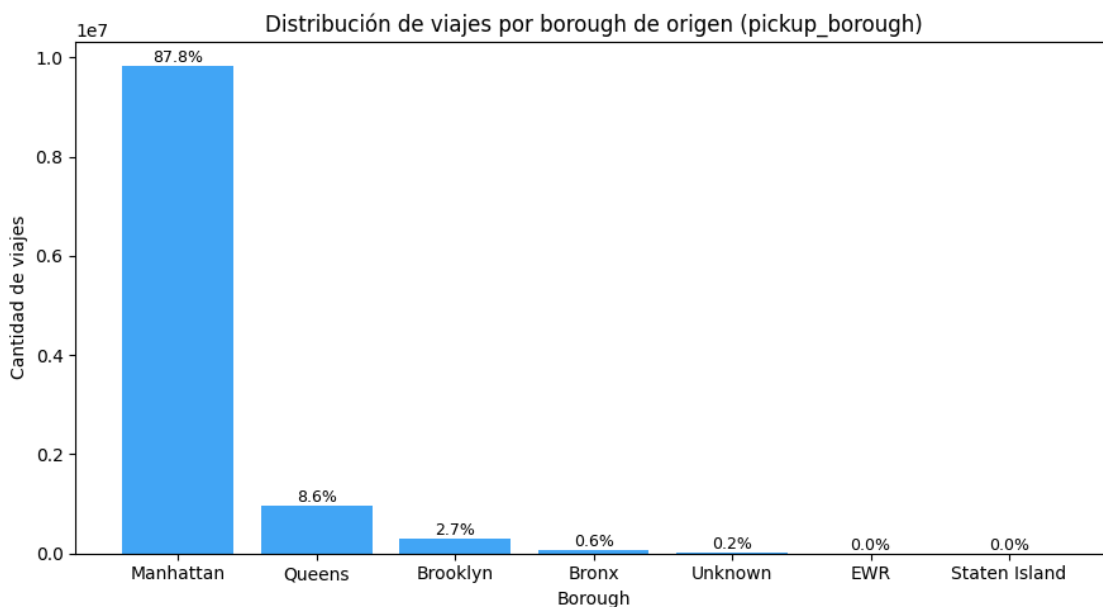
- **Métodos de pago y el sesgo de la propina:** La tarjeta de crédito domina como método de pago, seguida por el efectivo.

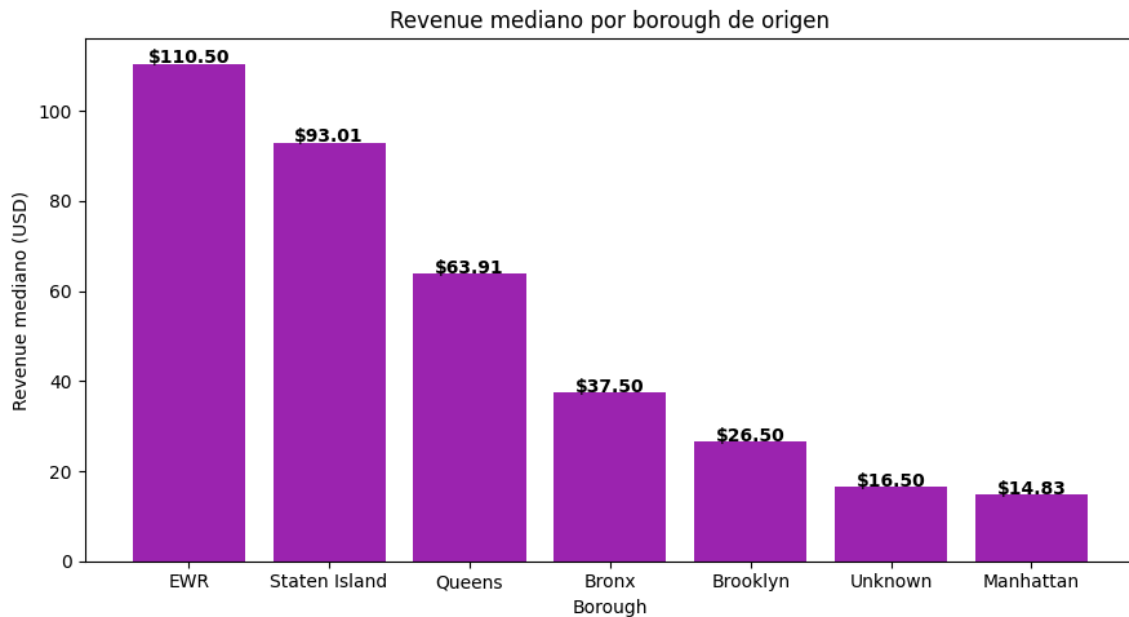
El dataset cuenta con la limitación de no registrar las propinas en efectivo. Al profundizar en la categoría de pago "Desconocido" (**Unknown**, 20.2%). Al analizar las propinas, se reveló que este "Desconocido" poseía un 91.57% de viajes sin propina registrada, un valor muy diferente al 6.1% de la tarjeta de crédito pero muy similar al pago en efectivo (99.95% sin propina). Esto nos inclina a creer (aunque no afirmar) que la mayoría de los pagos de categoría "Desconocido" son pagos en efectivo mal registrados.



- **Visualización de oferta y demanda:** Al analizar la distribución de los viajes por borough de origen se observa que Manhattan contiene la inmensa mayoría de los viajes con un 87.8%, lo cual es esperable al ser el centro comercial y turístico de NYC.

Sin embargo, al estudiar la ganancia mediana de los taxistas (variable **revenue** creada a partir de otras variables) por borough se ve a Manhattan como el último puesto, mientras que EWR (Un borough con una cantidad casi despreciable de viajes) toma el primer puesto por una amplia diferencia. Estos resultados pueden ser interpretados debido a la oferta y la demanda de los viajes por borough, en Manhattan la alta oferta desploma el precio mientras que en EWR pasa lo contrario.





4. Tratamiento de valores nulos

Con este panorama, se detallan los tratamientos de los datos faltantes:

- **Eliminación de la columna `store_and_fwd_flag`:** Inicialmente, se hipotetizó que los valores nulos en esta variable respondían a fallas de conexión con el servidor durante viajes de larga distancia. No obstante, el análisis por rangos (bins) de distancia demostró que la tasa de faltantes se concentraba en viajes medios (3-10 millas) y caía en los extremos, descartando una relación monótona.

Al contrastar las tasas de nulos por zonas de origen (`PULocationID`) frente a las de destino (`DOLocationID`), se observó que la variabilidad de nulos estaba fuertemente condicionada por el punto de partida.

El cruce con `VendorID` demostró que los proveedores minoritarios (como Myle Technologies y Helix) presentaban tasas extremas de 100% y 0% de nulos, respectivamente, mientras que los mayoritarios mostraban tasas heterogéneas. Esto permitió concluir que la pérdida de datos respondía a un mecanismo MAR (Missing At Random), ya que la nulidad de la variable estudiada dependía de otras dos variables. Para evitar la introducción de sesgos sistemáticos en zonas particulares mediante una imputación forzada, se decidió eliminar la columna por completo.

- **Imputación de `Airport_fee`:** El tratamiento de esta variable se abordó mediante el cruce geográfico utilizando el catálogo `taxi_zone_lookup.csv`. Se aislaron los identificadores correspondientes a los aeropuertos JFK y LaGuardia y se imputó el valor oficial de 1.75 USD exclusivamente a los registros nulos cuyo origen coincidiera con estas zonas aeroportuarias, asignando 0 USD al resto de los valores faltantes.

- **Imputación de `congestion_surcharge`:** Esta variable, que representa un cargo fijo de 2.50 USD para taxis amarillos en zonas de alto tránsito. Se identificaron los distritos de Manhattan ubicados al sur de la calle 96 (área de congestión estipulada por la TLC) y se imputó a todos los registros nulos o inválidos que tuvieran su punto de origen o destino dentro de este perímetro el cargo de 2.50 USD, mientras que a los viajes fuera de la zona se les asignó 0 USD. Estos valores son los dos únicos posibles según la regulación de la TLC.
- **Imputación de `passenger_count` y `RatecodeID`:** Para la cantidad de pasajeros, al ser una variable estrictamente cuantitativa discreta, se optó por imputar los valores nulos utilizando la moda. Usar la media habría generado valores decimales que no tienen sentido por ser una variable discreta.

Bajo el mismo criterio los faltantes y valores inválidos (como el código 99) en la variable cualitativa nominal `RatecodeID` se reemplazaron con la moda. Dado que la distribución de esta variable está masivamente concentrada en la tarifa estándar (`RatecodeID` 1).

5. Detección y Tratamiento de Valores Atípicos (Outliers)

La limpieza de outliers se ejecutó mediante una estrategia de dos etapas para equilibrar la pureza de los datos y la preservación de la realidad del negocio.

1. **Filtro por lógica de negocio:** se aplicó un filtrado sobre valores físicamente imposibles o que violaban la regulación. Esto incluyó depurar fechas fuera del trimestre objetivo, viajes con duraciones negativas o superiores a 12 horas, cantidades de pasajeros fuera del límite legal (1 a 6), distancias de más de 100 millas, tarifas base inferiores al monto mínimo legal (\$3.00 USD) y montos totales fuera del rango \$3.00–\$500 USD. Esta etapa removió aproximadamente 1.21 millones de registros anómalos.
2. **Análisis estadístico (IQR, Z-Score y Z-Score Modificado):** sobre la base pre-limpiada, se calcularon estas métricas sobre las variables `fare_amount`, `trip_distance`, `tip_amount`, `total_amount` y `trip_duration`. Los resultados arrojaron un alto volumen de atípicos estadísticos en todas ellas, consecuencia de su extrema asimetría positiva. Dado que los viajes extensos o altamente costosos (como los traslados desde/hacia aeropuertos) constituyen una proporción minoritaria pero completamente válida de la operación, se resolvió conservar dichos registros. Vale destacar además que `tip_amount` presenta una particularidad: al no registrar propinas en efectivo, una proporción importante de viajes tiene valor cero, lo que desplaza la mediana hacia abajo y hace que cualquier propina moderada aparezca como outlier estadístico. Filtrar estas variables con estos métodos estadísticos habría borrado una gran cantidad de valores válidos, alterando la representación real de los datos.

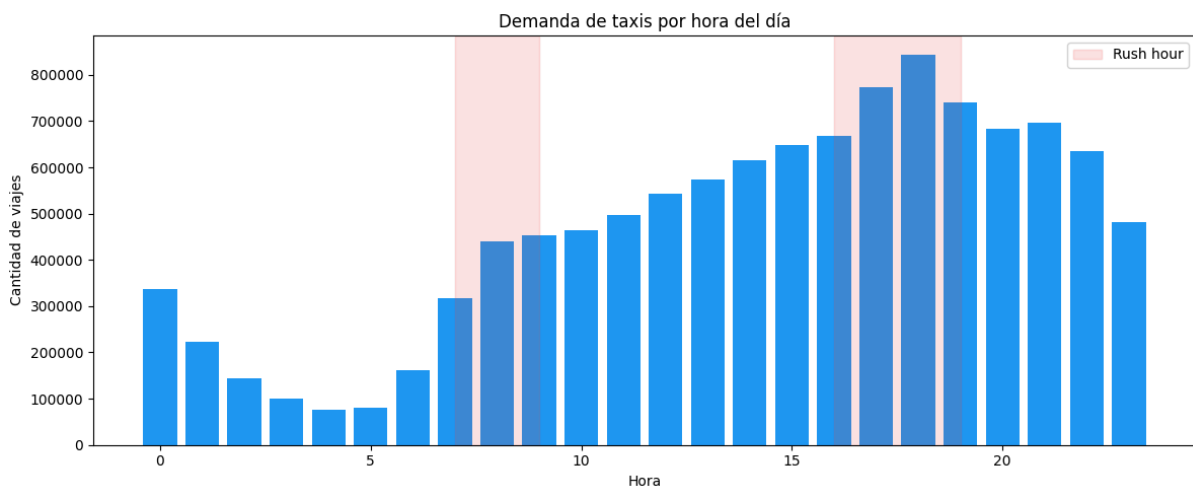
6. Generación de Nuevos Features

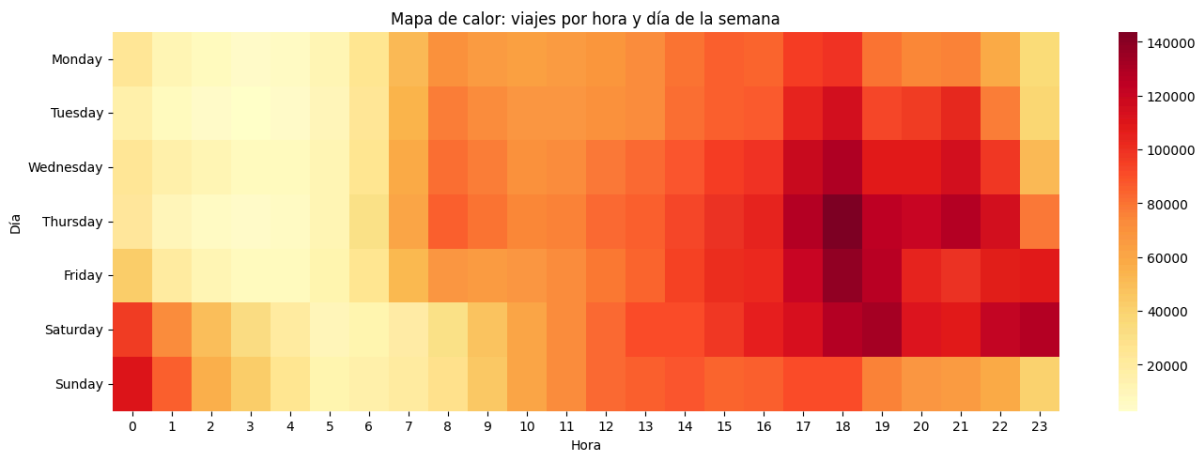
Para potenciar las capacidades exploratorias y predictivas, se crearon variables adicionales a partir de las que ya se tienen:

- **Temporales derivadas:** `hour`, `day_of_week`, `month`, `is_weekend` y `rush_hour` (definida operativamente de lunes a viernes en las franjas horarias 7–9 hs y 16–19 hs inclusive, según los patrones reales de congestión de NYC).
- **Métricas de Desempeño Operativo:** Se derivó la velocidad media en millas por hora (`speed_mph`), el gasto total por persona (`total_per_passenger`) y la tasa porcentual de propina sobre tarifa base ($\text{tip_rate} = \text{tip_amount} / \text{fare_amount}$). Esta última se calculó exclusivamente sobre los viajes abonados con tarjeta de crédito, ya que las propinas no se miden correctamente en los viajes pagados con efectivo.

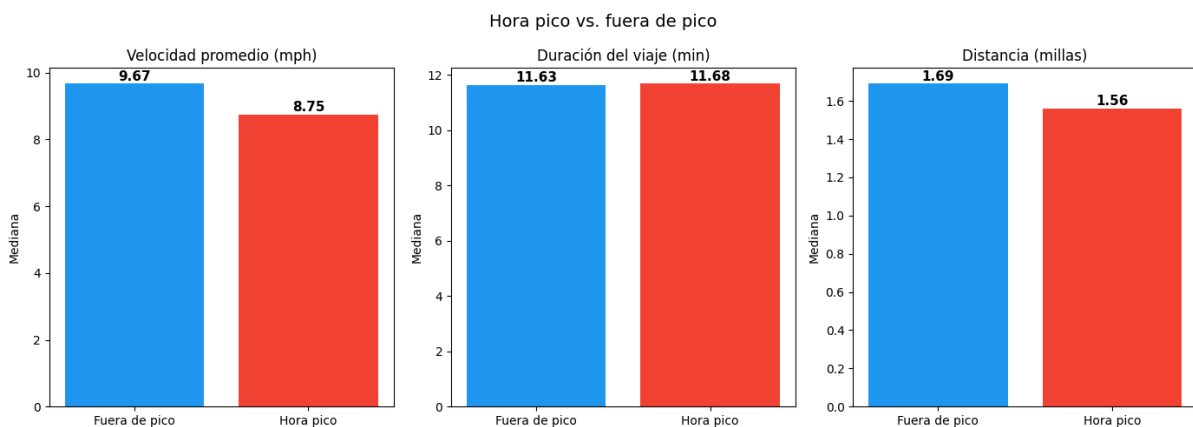
7. Visualizaciones y Respuestas a Preguntas de Investigación

Variabilidad temporal de la demanda: Los gráficos de barras mostraron una profunda caída de la actividad en la madrugada (03:00 - 05:00 hs). Si bien existe un pico de uso matutino de lunes a viernes, este se ve ampliamente superado por el repunte vespertino, alcanzando su punto máximo absoluto a las 18:00 hs con más de 840,000 viajes. El análisis por heatmap expuso que el sábado rompe la tendencia laboral y concentra el mayor volumen de la semana (1.83 millones de viajes), provocado por consumo recreativo prolongado hasta altas horas de la noche. Por ende, la hipótesis se confirma parcialmente: el pico vespertino apareció como se esperaba, pero el sábado superó a todos los días hábiles, lo cual no era esperado.



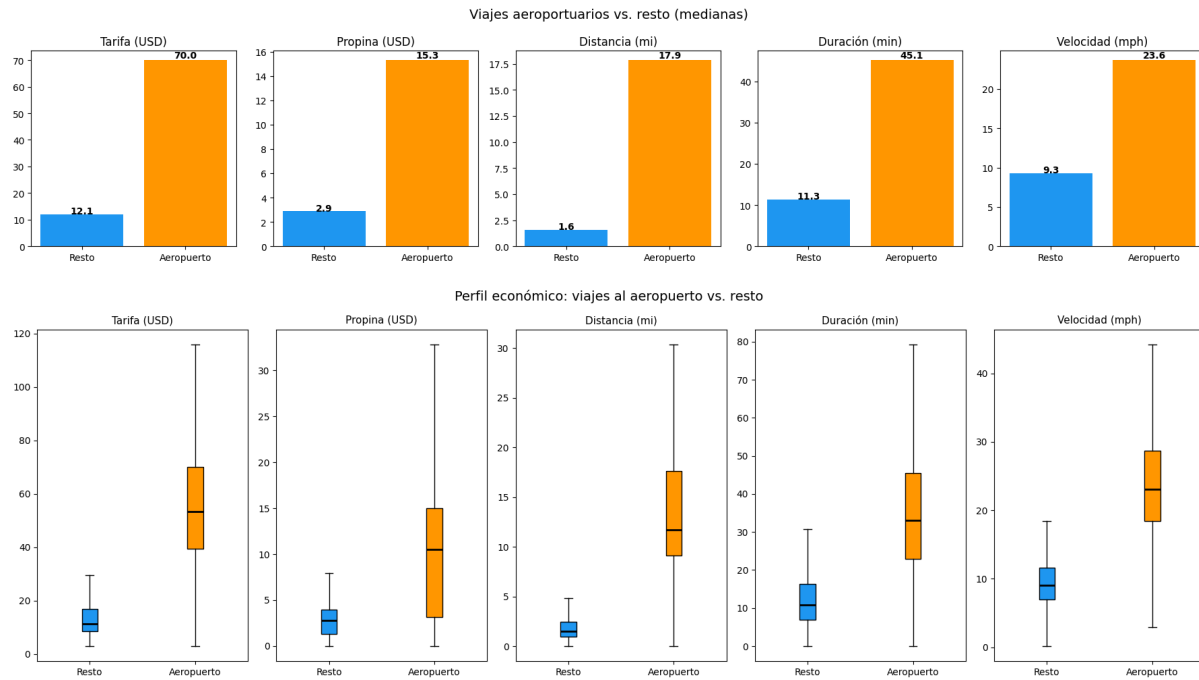


Impacto de la hora pico (Rush Hour): Al segmentar los viajes, se confirmó la penalización operativa durante la congestión. La velocidad mediana decae casi un 11% (de 9.67 a 8.75 mph), mientras que la longitud mediana del recorrido tiende a ser menor (1.69 vs. 1.56 millas) en lapsos de tiempo idénticos. Esto sugiere que en hora pico la flota se aboca a distancias más cortas y lentas dentro de embotellamientos urbanos.



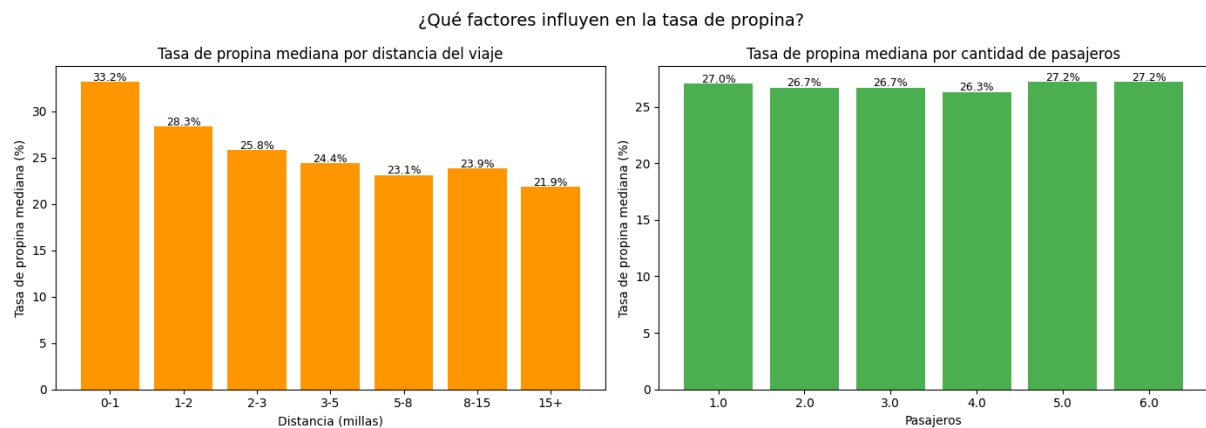
Rentabilidad por zona (Revenue = Fare + Tip): Cómo se desarrolló en el apartado 3, cuando se menciona la visualización de oferta y demanda. El aeropuerto de Newark se ubica a la cabeza generando ingresos medianos de 110 USD por viaje, seguido por las zonas exteriores a la ciudad. Aunque Manhattan prácticamente monopoliza los viajes, registra la rentabilidad más baja de la muestra (14.83 USD)

El perfil de los viajes al aeropuerto: Se validó la fuerte asimetría de esta categoría. Un traslado aeroportuario promedio incrementa la tarifa base más de cinco veces respecto a un viaje ciudadano regular (70 USD frente a 13 USD). Asimismo, registran la mayor longitud de tramo, tienen la mayor cantidad de propina y se desplazan a mayor velocidad porque la mayor parte del viaje es en autopista. La hipótesis se confirma



Factores de influencia sobre la propina: El cruce de la variable `tip_rate` refutó la hipótesis inicial. La propina decrece monótonamente a medida que el viaje se vuelve más largo, cayendo de un 33.2% en trayectos menores a 1 milla hasta consolidarse en un 21.9% para viajes de 15 millas o más.

Por otro lado, la cantidad de ocupantes en el vehículo probó no tener correlación, siendo estable en torno al 26-27% para cualquier cantidad de pasajeros.



EJ2: Modelos de Clasificación Binaria

1. Descripción del dataset y clasificación de variables

El dataset utilizado corresponde a observaciones meteorológicas diarias registradas en múltiples estaciones de Australia durante aproximadamente 10 años, obtenido de Kaggle. Para este ejercicio se filtraron únicamente las estaciones pertenecientes a tres territorios: Queensland, Nueva Gales del Sur y el Territorio de la Capital Australiana. Tras el filtrado, el dataset cuenta con 51.235 filas y 23 columnas. Cada fila representa las observaciones climáticas de un día en una estación particular.

Variables del dataset

Variables cuantitativas continuas

- `MinTemp` / `MaxTemp`: temperaturas mínima y máxima del día (°C).
- `Rainfall`: precipitación registrada (mm).
- `Evaporation` / `Sunshine`: evaporación (mm) y horas de sol brillante.
- `WindGustSpeed` / `WindSpeed9am` / `WindSpeed3pm`: velocidades de viento (km/h).
- `Humidity9am` / `Humidity3pm`: humedad relativa a las 9am y 3pm (%).
- `Pressure9am` / `Pressure3pm`: presión atmosférica a nivel del mar (hPa).
- `Cloud9am` / `Cloud3pm`: fracción del cielo cubierta por nubes (0–8 oktas).
- `Temp9am` / `Temp3pm`: temperatura a las 9am y 3pm (°C).

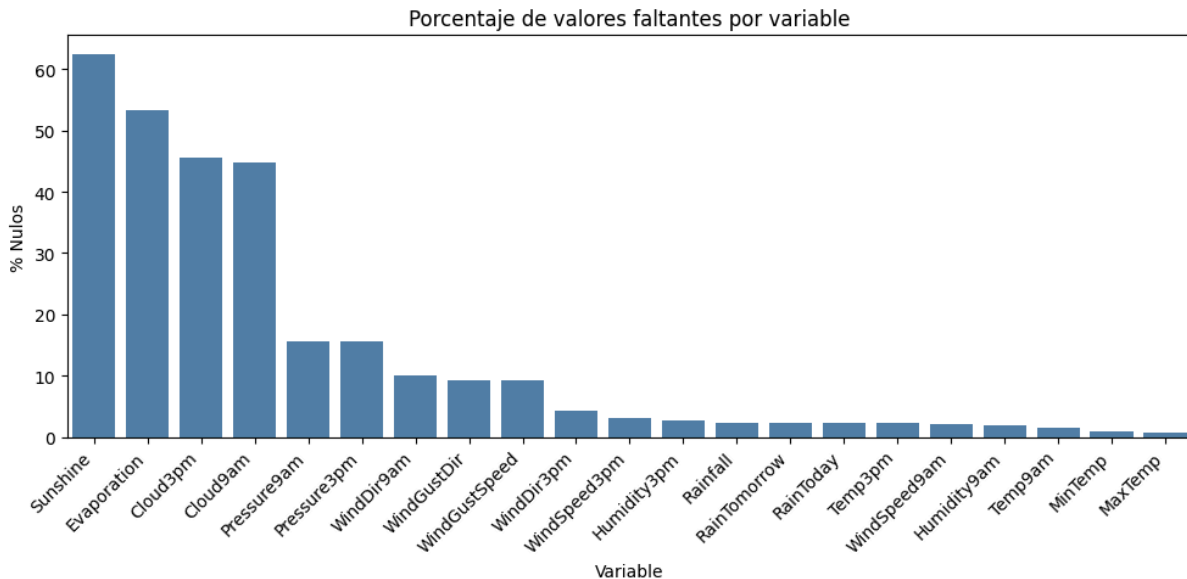
Variables categóricas

- `Location`: nombre de la estación meteorológica.
- `Date`: fecha de la observación (formato `object`, convertida a `datetime`).
- `WindGustDir` / `WindDir9am` / `WindDir3pm`: dirección del viento (16 puntos cardinales).
- `RainToday`: variable binaria (1 si la precipitación supera 1 mm, 0 en caso contrario).
- `RainTomorrow`: variable objetivo binaria (Yes/No). Indica si llovió al día siguiente.

El dataset presenta un claro desbalance en la variable objetivo: el 78,5% de los registros corresponden a días sin lluvia (clase 0) y el 21,5% a días con lluvia (clase 1). Este desbalance fue tratado explícitamente en todos los modelos mediante pesos balanceados (`class_weight='balanced'`), priorizando el recall como métrica de evaluación principal.

2. Tratamiento de valores faltantes

Se analizó el porcentaje de nulos por variable. Presentado en el siguiente gráfico:



A partir de estos resultados las decisiones tomadas fueron:

- **Sunshine (62,4%)** y **Evaporation (53,3%)**: eliminadas por superar el umbral del 50%. Imputar más de la mitad de los datos introduciría demasiado ruido sobre la información.
- **Cloud9am (44,8%)** y **Cloud3pm (45,7%)**: imputadas con la mediana por estación (Location).
- **Resto de variables numéricas (< 16% de nulos)**: imputadas con la mediana por estación. Se usó la mediana local porque el clima varía significativamente entre regiones.
- **Casos especiales**: algunas estaciones (Newcastle, Penrith, MountGinini, BadgerysCreek, NorahHead, Tuggeranong) tenían columnas completamente vacías para ciertas variables. Se aplicó un segundo paso de imputación con la mediana global como fallback, resultando en 0 nulos residuales.
- **RainToday y RainTomorrow (2,4%)**: se eliminaron las filas con nulos, ya que no es válido imputar la variable objetivo ni su análoga del día actual.
- **Variables de dirección del viento (WindGustDir, WindDir9am, WindDir3pm)**: imputadas con la moda por estación. Para Newcastle (sin ningún valor válido de WindGustDir) se aplicó un fallback con la moda global.

3. Detección y Tratamiento de Valores Atípicos (Outliers)

Se verificaron los rangos de las variables numéricas clave. Los valores extremos encontrados (como lluvia máxima de 371 mm y ráfagas de hasta 135 km/h), aunque no muy comunes, son físicamente posibles de suceder en zonas tropicales como Queensland.

No se eliminaron registros por outliers, ya que los modelos de árbol que se utilizarán son robustos frente a este tipo de valores extremos al trabajar con particiones binarias.

4. Feature Engineering, Encoding y Normalización

Para aportar señal predictiva adicional se crearon variables derivadas:

- **Month**: mes de la observación (extraído de Date).
- **Season**: estación del año (Verano, Otoño, Invierno, Primavera), mapeada desde el mes.
- **TempRange**: rango térmico diario (MaxTemp - MinTemp). Captura la amplitud térmica del día.
- **HumidityDrop**: caída de humedad entre mañana y tarde (**Humidity9am** - **Humidity3pm**). Una caída pronunciada suele indicar condiciones más secas por la tarde.
- **PressureDrop**: caída de presión entre mañana y tarde (**Pressure9am** - **Pressure3pm**). Una presión descendente es un indicador de posible inestabilidad atmosférica y posible lluvia.

	TempRange	HumidityDrop	PressureDrop	Month
count	49143.00	49143.00	49143.00	49143.00
mean	11.18	18.73	2.62	6.39
std	4.92	17.00	1.73	3.43
min	-2.90	-62.00	-17.20	1.00
25%	7.30	7.00	1.80	3.00
50%	10.80	19.00	2.60	6.00
75%	14.80	30.00	3.60	9.00
max	32.00	83.00	17.60	12.00

En la parte de encoding de las variables, separamos dos grupos sobre los que se hizo un tratamiento:

- **Encoding binario**: **RainToday** y **RainTomorrow** convertidas a 0/1 mediante comparación directa con 'Yes'.
- **Label Encoding**: variables categóricas **WindGustDir**, **WindDir9am**, **WindDir3pm**, **Location** y **Season** codificadas con LabelEncoder de sklearn.

Si bien los modelos de Árbol de decisión y Random Forest son invariantes a la escala de las variables, se realiza una estandarización (StandardScaler) sobre las variables numéricas continuas como paso de preprocesamiento general. Esto se debe a que el Modelo 3 a elección puede requerir que las features estén en una escala comparable. Las variables categóricas codificadas (direcciones de viento, ubicación, estación) y las binarias (**RainToday_enc**) no se escalan ya que su codificación numérica no representa magnitud.

El conjunto de features final contiene 24 variables. La división train/test se realizó en proporción 80/20, resultando en 49.636 observaciones de entrenamiento y 12.410 de test.

5. Entrenamiento y evaluación: Árbol de Decisión

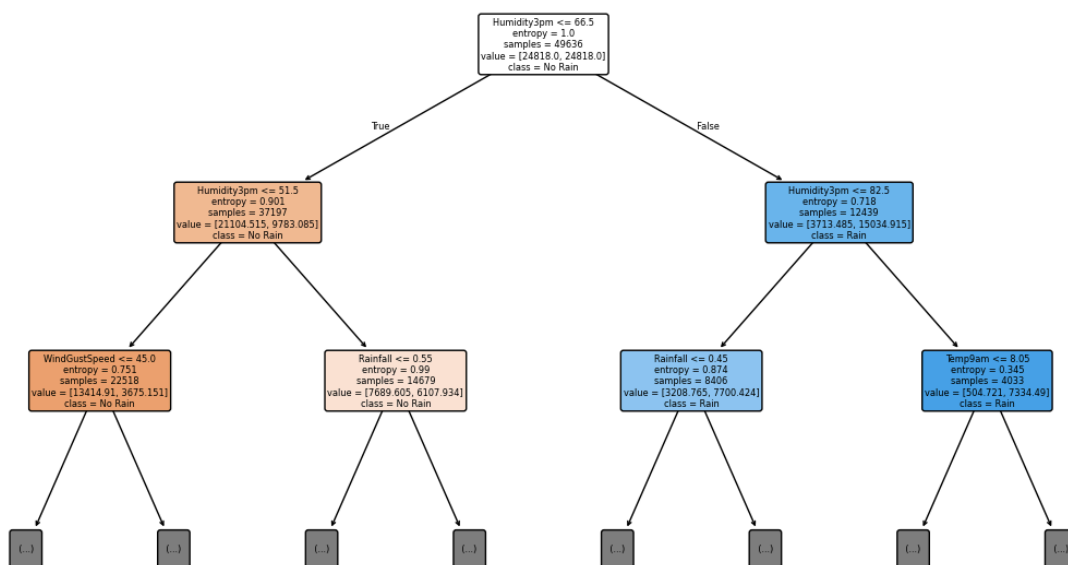
Se implementó un `DecisionTreeClassifier` de scikit-learn utilizando el criterio de entropía.

Se realizó una búsqueda mediante `GridSearchCV` con validación cruzada de 5 folds, optimizando el `F1-score` y aplicando `class_weight='balanced'`, los mejores hiperparámetros fueron: `ccp_alpha=0.0`, `max_depth=10`, `min_samples_leaf=20`.

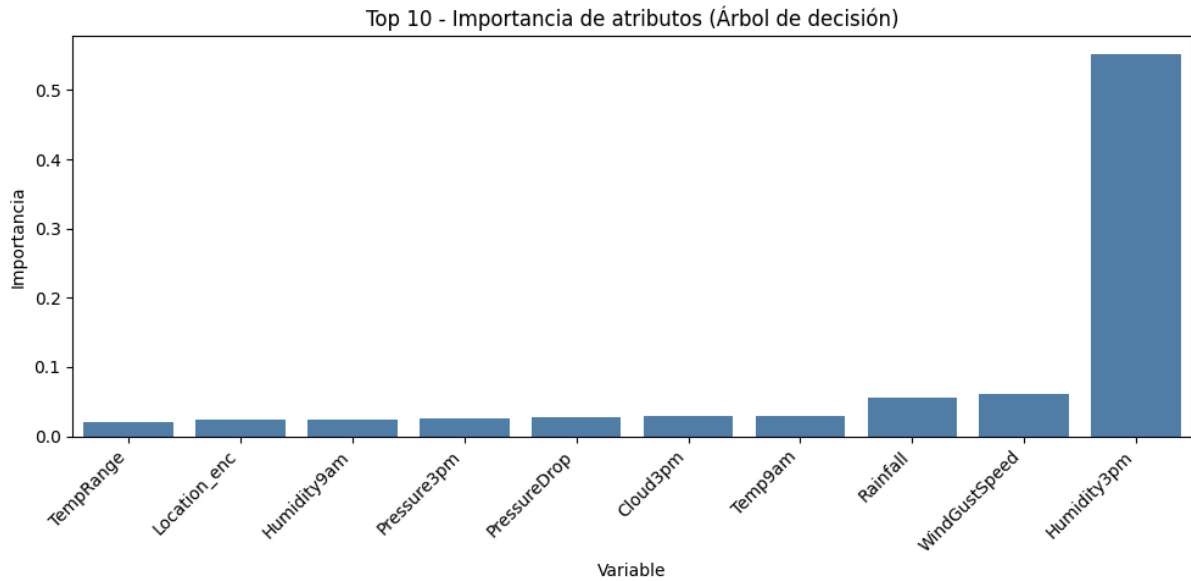
El modelo final muestra reglas meteorológicamente coherentes, siendo las primeras separaciones del árbol las siguientes:

- Nodo raíz - `Humidity3pm ≤ 66,5`: la humedad relativa de la tarde es el factor más discriminante. Días con humedad baja ($\leq 66,5\%$) son clasificados como 'No llueve'; días con humedad alta van a la rama de lluvia.
- Rama izquierda (humedad baja): se evalúa `Humidity3pm ≤ 51,5` y `WindGustSpeed` para refinar la predicción de 'No llueve'.
- Rama derecha (humedad alta): se evalúa `Humidity3pm ≤ 82,5` en los casos positivos se prosigue con `Rainfall ≤ 0,45`, en los negativos `Temp9am ≤ 7,65`. Temperaturas bajas con alta humedad indican lluvia casi segura.

Árbol de Decisión final (primeros 2 niveles)



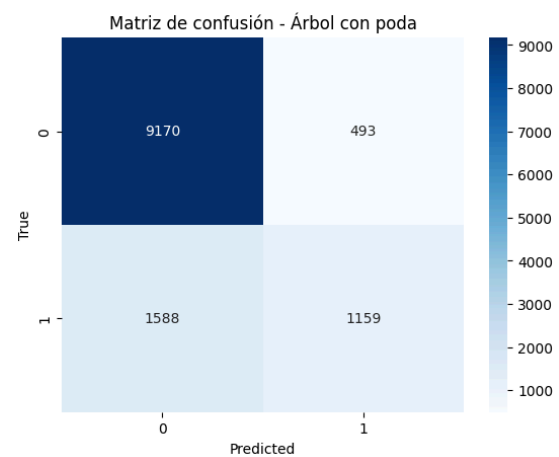
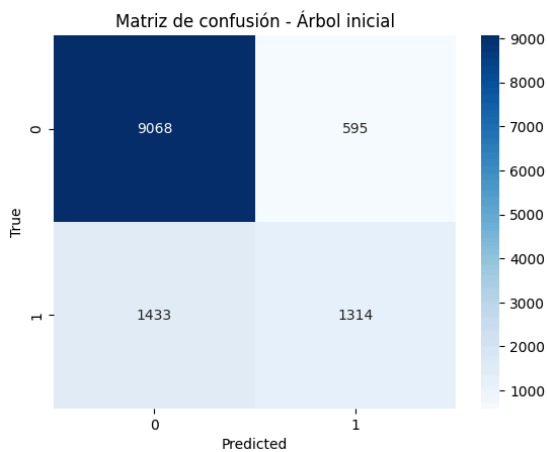
Analizamos cuáles fueron las 10 variables más importantes para que el árbol de decisión separe los casos entre 'No Rain' y 'Rain':

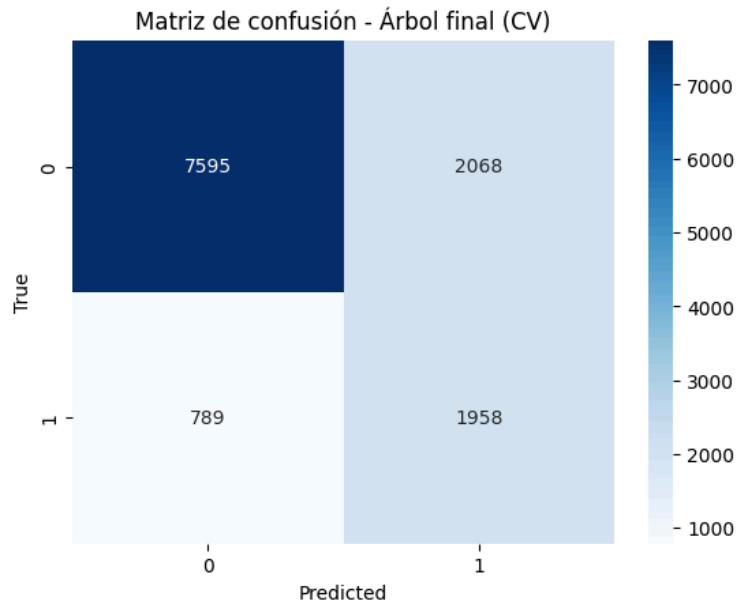


La variable más importante según el modelo es **Humidity3pm** (importancia = 0.55), seguida de **WindGustSpeed** (0.065) y **Rainfall** (0.047). Esto es consistente con la lógica meteorológica: la alta humedad vespertina es el principal precursor de lluvia al día siguiente.

Evaluación del modelo (matrices de confusión)

En cuanto a las matrices de confusión, se analizó el progreso desde el árbol inicial hasta el optimizado.





Modelo	Accuracy	Precision	Recall	F1-Score	Parámetros clave
Árbol inicial (sin optimizar)	0.838	0.688	0.478	0.564	{‘max_depth’: 10, ‘criterion’: ‘entropy’, ‘min_samples_leaf’: 10}
Árbol con poda (class_weight)	0.832	0.701	0.421	0.526	{‘ccp_alpa’: 0.001, ‘criterion’: ‘entropy’, ‘max_depth’: 10, ‘min_samples_leaf’: 10}
Árbol final (GridSearchCV)	0.769	0.486	0.712	0.578	{‘ccp_alpha’: 0.0, ‘criterion’: ‘entropy’, ‘max_depth’: 10, ‘min_samples_leaf’: 20, ‘class_weight’: ‘balanced’}

El árbol optimizado alcanzó en test un Accuracy de 0.769, Precision de 0.486, Recall de 0.712 y F1-Score de 0.578. Se prioriza el recall (detecta 2 de cada 3 días de lluvia reales), asumiendo el trade-off de menor precisión y accuracy, dado que en contexto meteorológico no detectar una lluvia real es más costoso que una falsa alarma.

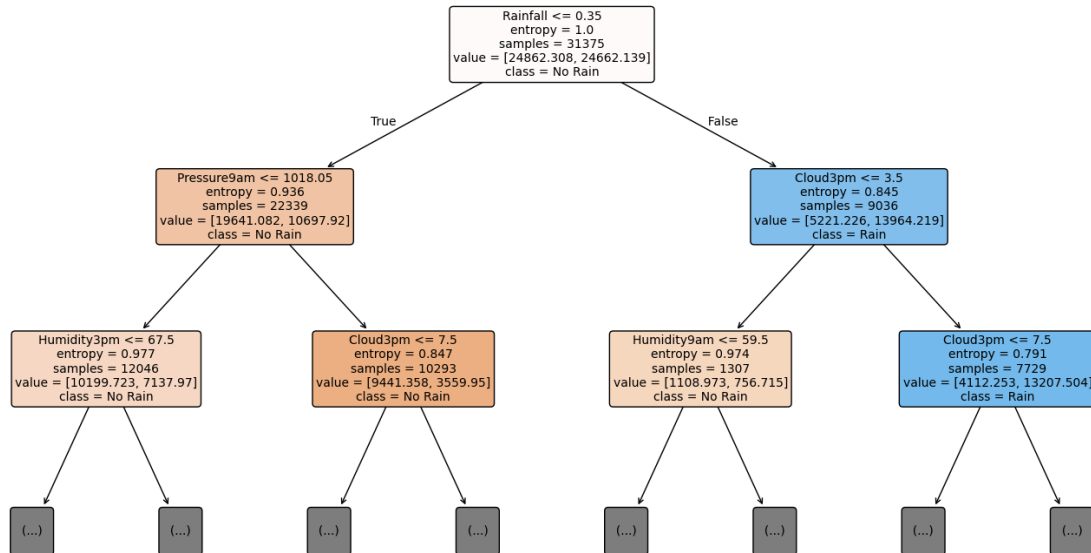
6. Entrenamiento y evaluación: Random Forest

Se implementó un `RandomForestClassifier` de scikit-learn, que entrena múltiples árboles de decisión sobre submuestras aleatorias del conjunto de entrenamiento y, en cada nodo, considera sólo un subconjunto aleatorio de features. La predicción final es la votación mayoritaria de todos los árboles.

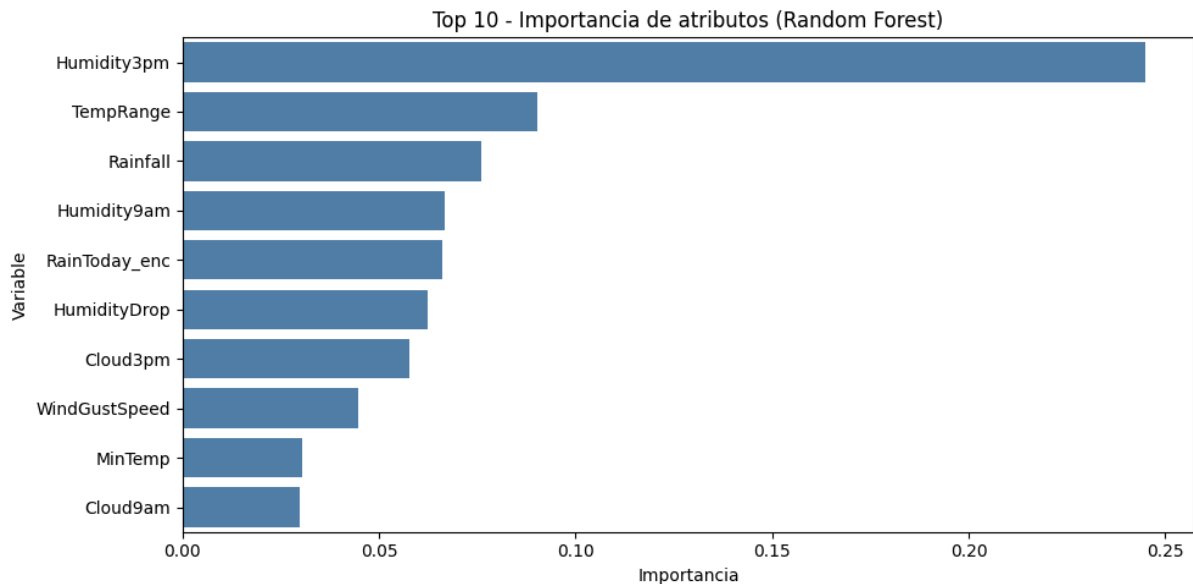
La búsqueda de hiperparámetros (`RandomizedSearchCV`) prioriza el recall, obteniendo como mejores parámetros: `n_estimators=100`, `max_depth=10`, `min_samples_split=5`, `min_samples_leaf=1`, `class_weight='balanced'`.

Al analizar un árbol representativo del ensamble, se observa que la división inicial parte de variables distintas (como **Rainfall** en lugar de **Humidity3pm**), lo que ilustra el mecanismo central del Random Forest: al forzar subconjuntos aleatorios de variables, los árboles se correlacionan entre sí.

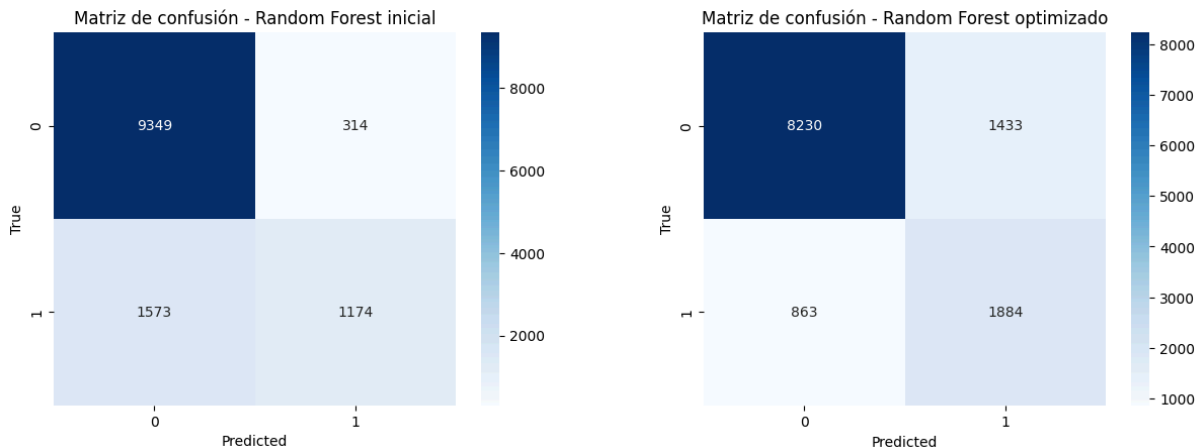
Árbol representativo del Random Forest (primeros 2 niveles)



Se estudian las variables más importantes según el modelo de Random Forest



Evaluación del modelo (matrices de confusión)



Modelo	Accuracy	Precision	Recall	F1-Score	Parámetros clave
RF inicial (sin optimizar)	0.847	0.788	0.427	0.554	{'n_estimators': 100, 'criterion': 'entropy', 'max_depth': 10, 'min_samples_leaf': 10, 'random_state': 42, 'n_jobs': 1}
RF optimizado (RandomizedSearchCV)	0.814	0.567	0.685	0.621	{'n_estimators': 100, 'criterion': 'entropy', 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 10, 'max_features': 'sqrt', 'class_weight': 'balanced'}

El Random Forest optimizado mejoró sustancialmente el recall respecto al modelo base (de 0.427 a 0.685), aunque con una caída en precisión. El F1-score subió de 0.554 a 0.62, siendo el mejor resultado hasta el momento. La accuracy descendió ligeramente (de 0.85 a 0.82) al balancear mejor las clases. Comparado con el árbol individual, el Random Forest logra mayor recall y F1, al aprovechar la diversidad de múltiples árboles para reducir la varianza.

7. Entrenamiento y evaluación: XGBoost

Se eligió XGBoost como tercer modelo por las siguientes razones:

- **Familia distinta:** representa la familia de boosting (árbol individual → bagging → boosting), completando una comparación significativa entre tres paradigmas de modelado.
- **Mejor trade-off sesgo-varianza:** el boosting secuencial prioriza los ejemplos mal clasificados, lo que en la práctica genera mayor precisión predictiva.

- **Robustez:** reduce el riesgo de overfitting respecto al Gradient Boosting clásico y es naturalmente robusto frente a outliers sin requerir normalización de features.
- **Manejo explícito del desbalance:** el parámetro `scale_pos_weight` permite compensar explícitamente el desbalance de clases, penalizando más los errores sobre la clase minoritaria durante el entrenamiento.

Se realizó una búsqueda con `GridSearchCV` (3 folds, `scoring='f1'`) con `scale_pos_weight=neg/pos` (≈ 3.5) para compensar el desbalance. La grilla explorada fue:

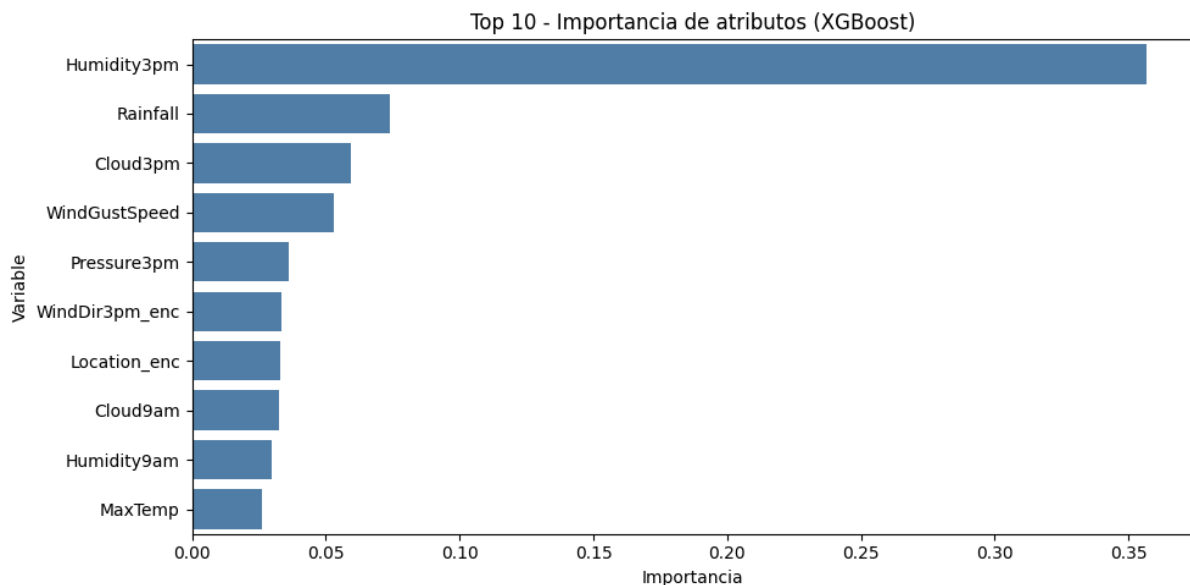
- `max_depth`: [3, 5]
- `learning_rate`: [0.05, 0.1]
- `n_estimators`: [100, 200]

Los mejores hiperparámetros obtenidos fueron: `learning_rate=0.1`, `max_depth=5`, `n_estimators=200`, `scale_pos_weight=3.5`.

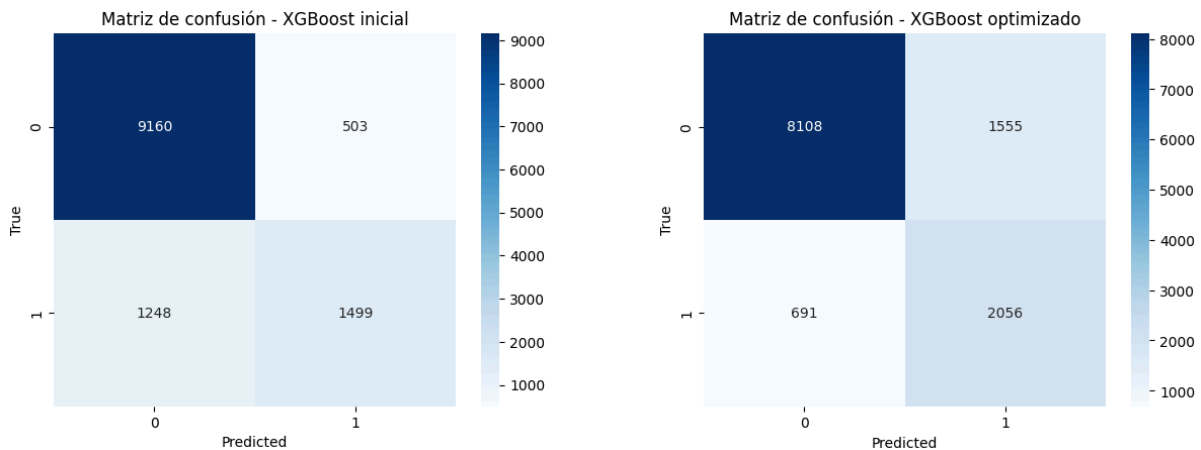
El mejor F1 en CV fue 0.646.

El modelo inicial (sin `scale_pos_weight`) alcanzó un recall de 0.525 para la clase lluvia, evidenciando que sin corrección tiende a predecir mayormente 'No llueve'. Al incorporar `scale_pos_weight=3.5`, el recall subió a 0.745, demostrando que el desbalance de clases era el principal limitante del modelo base. Esto es equivalente al `class_weight='balanced'` utilizado en los modelos de árbol y Random Forest.

Se estudian las variables más importantes según el modelo de XGBoost:



Evaluación del modelo (matrices de confusión)



Modelo	Accuracy	Precision	Recall	F1-Score	Parámetros clave
XGBoost inicial (sin scale_pos_weight)	0.858	0.748	0.545	0.631	{'random_state': 42, 'eval_metric': 'logloss', 'n_jobs': 1}
XGBoost optimizado (GridSearchCV)	0.819	0.569	0.748	0.646	{'random_state': 42, 'eval_metric': 'logloss', 'n_jobs': 1, 'learning_rate': 0.1, 'max_depth': 5, 'n_estimators': 200, 'scale_pos_weight': 'yes/neg'}

El XGBoost optimizado logra el recall más alto de los tres modelos (0.748), detectando correctamente 3 de cada 4 días de lluvia real. La precisión es similar al Random Forest (0.567) y el F1-score es el más alto (0.646). La accuracy es comparable a la del Random Forest (0.82).

8. Comparación de Modelos

Modelo	F1	Precision	Recall	Accuracy
Árbol de decisión (optimizado)	0.578	0.486	0.712	0.769
Random Forest (optimizado)	0.621	0.567	0.685	0.814
XGBoost (optimizado)	0.646	0.569	0.748	0.819

Los tres modelos muestran perfiles de performance claramente diferenciados. El árbol de decisión optimizado obtiene el mayor accuracy entre los extremos pero el F1 y recall son más bajos, lo que refleja que tiende a clasificar con mayor conservadurismo la clase

minoritaria. El Random Forest mejora todos los indicadores al combinar múltiples árboles descorrelacionados, reduciendo la varianza y aprovechando señales predictivas distribuidas entre variables.

XGBoost resulta el modelo más adecuado para este problema. Alcanza el recall más alto (0.748) y el mejor F1 (0.646), que son las métricas prioritarias dado el desbalance de clases (78%/22%) y el costo asimétrico de los errores, ya que en predicción meteorológica, no detectar un día de lluvia es más perjudicial que generar una alerta falsa. La menor precisión de XGBoost (0.569) es una consecuencia esperada de priorizar el recall mediante `scale_pos_weight`, y representa un trade-off aceptable en este contexto.

En cuanto al overfitting, los tres modelos muestran brechas similares entre train y test (6-7 puntos en F1), lo que indica que ninguno sobreajusta de forma significativa. El Random Forest presenta la brecha más amplia (7.3 puntos), seguido por el árbol de decisión (6.6) y XGBoost (6.5), aunque las diferencias son menores y no comprometen la validez de los modelos.

En síntesis, XGBoost ofrece el mejor balance entre capacidad predictiva y robustez, y es el modelo recomendado para el caso de uso planteado. Random Forest es una alternativa válida si se priorizan mayor precisión y un modelo más fácil de interpretar componente a componente.

EJ3: Modelos de Regresión

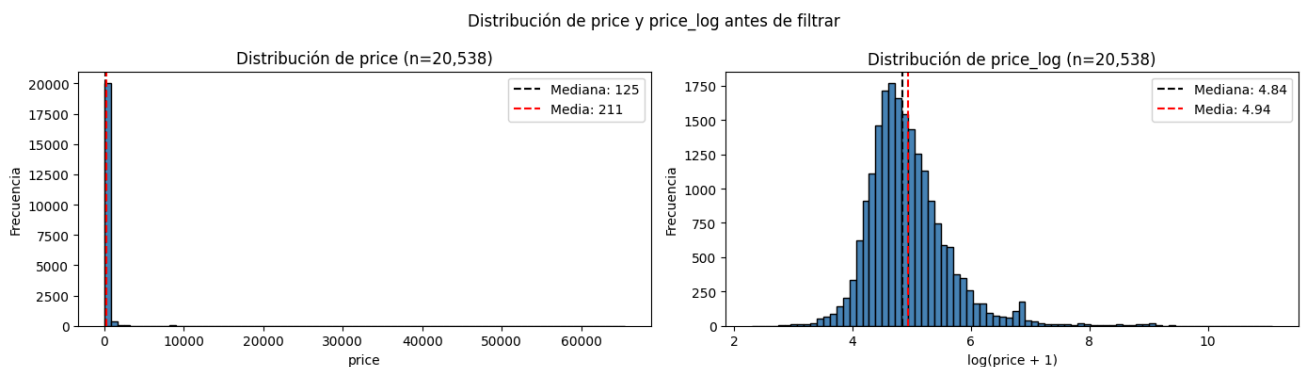
1. Breve descripción y análisis del dataset

El dataset proviene de la plataforma Inside Airbnb y contiene información detallada sobre alojamientos publicados en Milán, extraída el 22 de septiembre de 2025. El objetivo es predecir el precio de alquiler por noche (**price**) en función de las características del aviso. En su forma inicial, el dataset contaba con 22933 filas y 79 columnas, organizadas en cinco grandes grupos: atributos del host (perfil, historial, verificación), características del alojamiento (tipo de propiedad, capacidad, ubicación), restricciones de noches y disponibilidad, puntajes de reseñas, y variables de geolocalización.

2. Preparación de la variable target

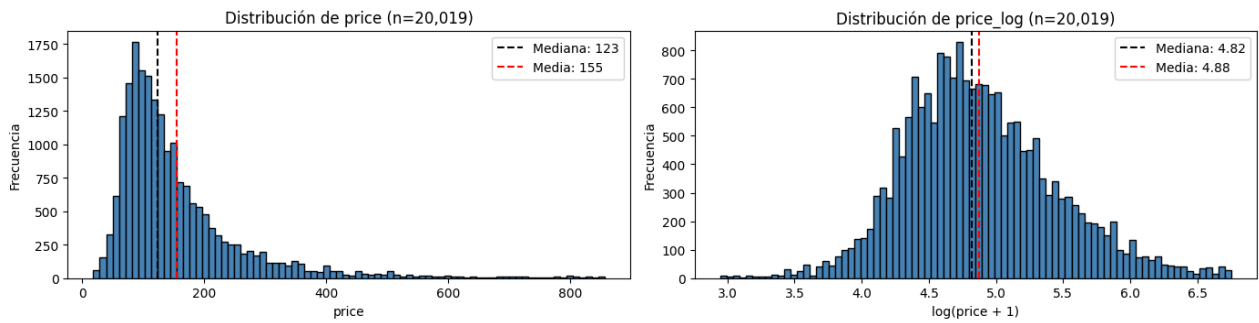
La variable **price** se encontraba codificada como string (por ejemplo, "\$125.00"), por lo que el primer paso fue eliminar los símbolos de moneda y convertirla a tipo float. A continuación se eliminaron las filas con **price** nulo, dado que al tratarse del target no es posible imputar ese valor. Finalmente, se creó una versión transformada **price_log = log1p(price)**.

Decidimos usar esta transformación por dos motivos. Por un lado, la distribución original de **price** presenta un sesgo muy grande a la derecha, con una media considerablemente superior a la mediana y valores extremos que distorsionan tanto la visualización de la distribución como el ajuste de los modelos. Por otro lado, optamos por trabajar en escala logarítmica ya que mejora el ajuste de la Regresión Lineal (que es más robusto con distribución simétrica del target) y hace más eficiente el posterior filtrado de valores atípicos.



Para detectar y eliminar precios atípicos se aplicó el **Z-score modificado** sobre **price_log**. Se usó un umbral de 3.5, eliminando únicamente los valores más extremos (errores de carga o propiedades completamente fuera del rango de mercado de Milán) y conservando registros con precios altos pero realistas, acorde a departamentos de lujo o suites de hotel.

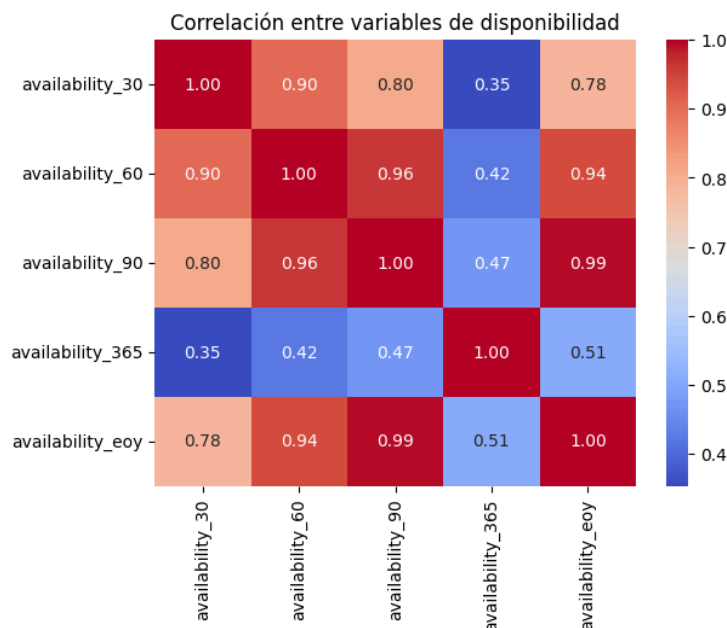
Distribución de price y price_log después de filtrar



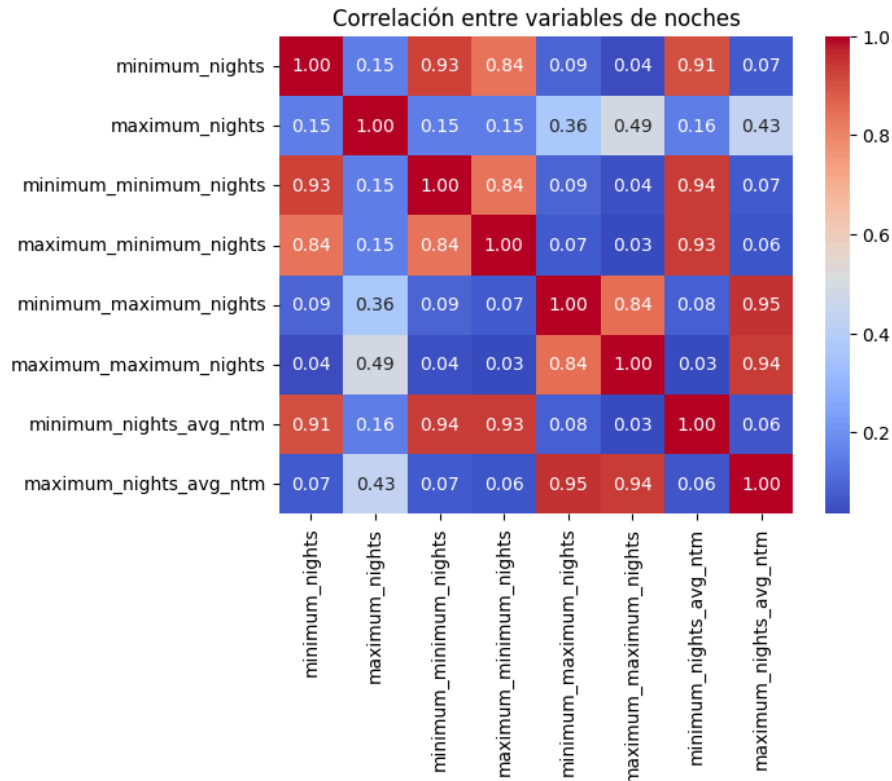
3. Filtrado de columnas irrelevantes

Antes de comenzar el análisis exploratorio se eliminaron las columnas que no aportan valor predictivo:

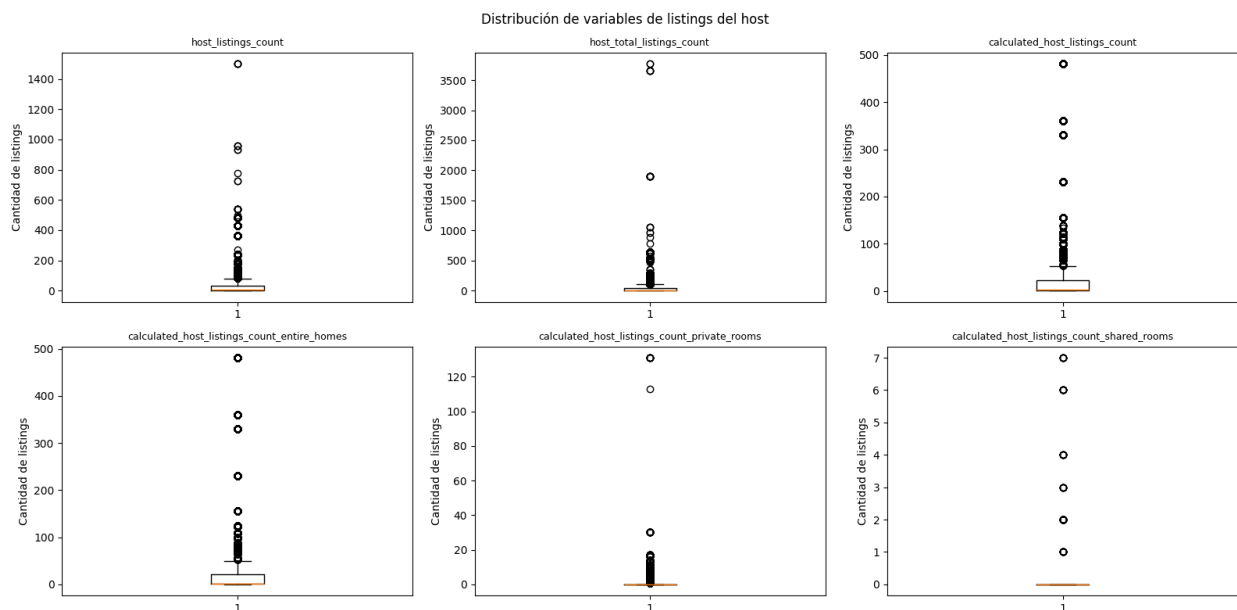
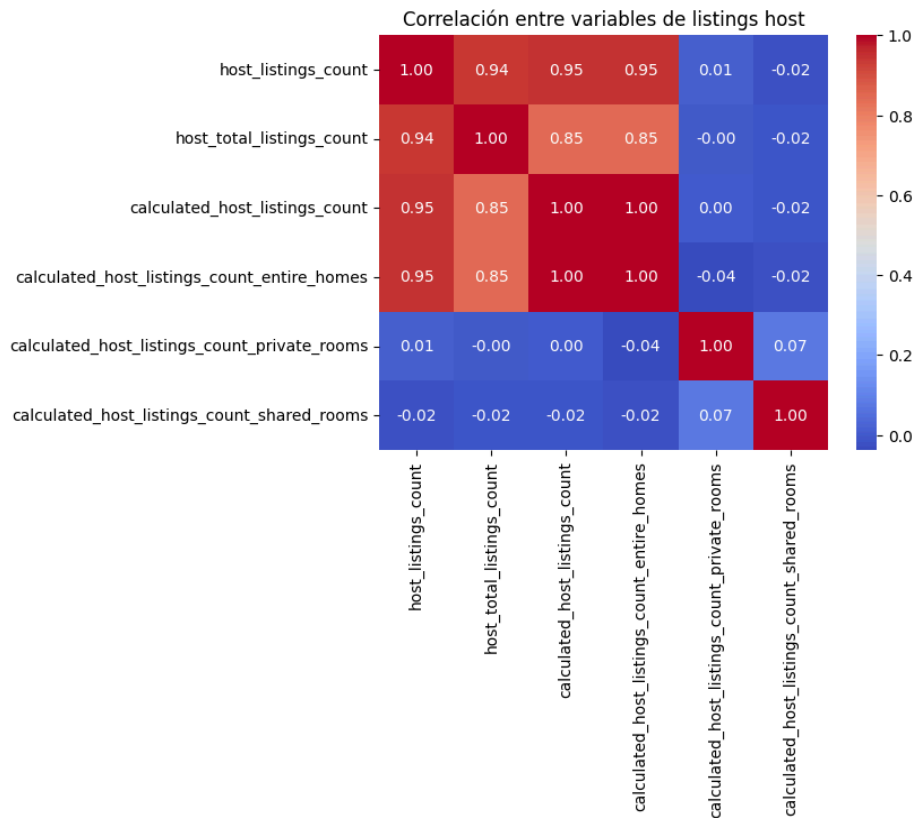
- **URLs e identificadores únicos** (`id`, `host_id`, `scrape_id`, `listing_url`, `picture_url`, y otras): no aportan características de los alojamientos.
- **Texto libre no procesado** (`name`, `description`, `neighborhood_overview`, `host_about`, `host_name`): su contenido queda fuera del alcance de este trabajo.
- **Columnas completamente vacías** (`neighbourhood_group_cleaned`, `calendar_updated`): nulas al 100%.
- `estimated_revenue_l365d`: eliminada para prevenir **data leakage**, ya que el revenue estimado se calcula a partir del precio, lo que significa incluir el target como feature.
- **Variables redundantes de disponibilidad**: se analizó la correlación entre las ventanas de disponibilidad (`availability_30`, `_60`, `_90`, `_365`, `_eoy`). Las primeras cuatro presentaron correlaciones de 0.80–0.99 entre sí, descartando `_60`, `_90` y `_eoy` por redundantes. Se conservaron `availability_30` y `availability_365` por representar horizontes temporales complementarios (corto y largo plazo) con correlación de apenas 0.35–0.47 entre sí.



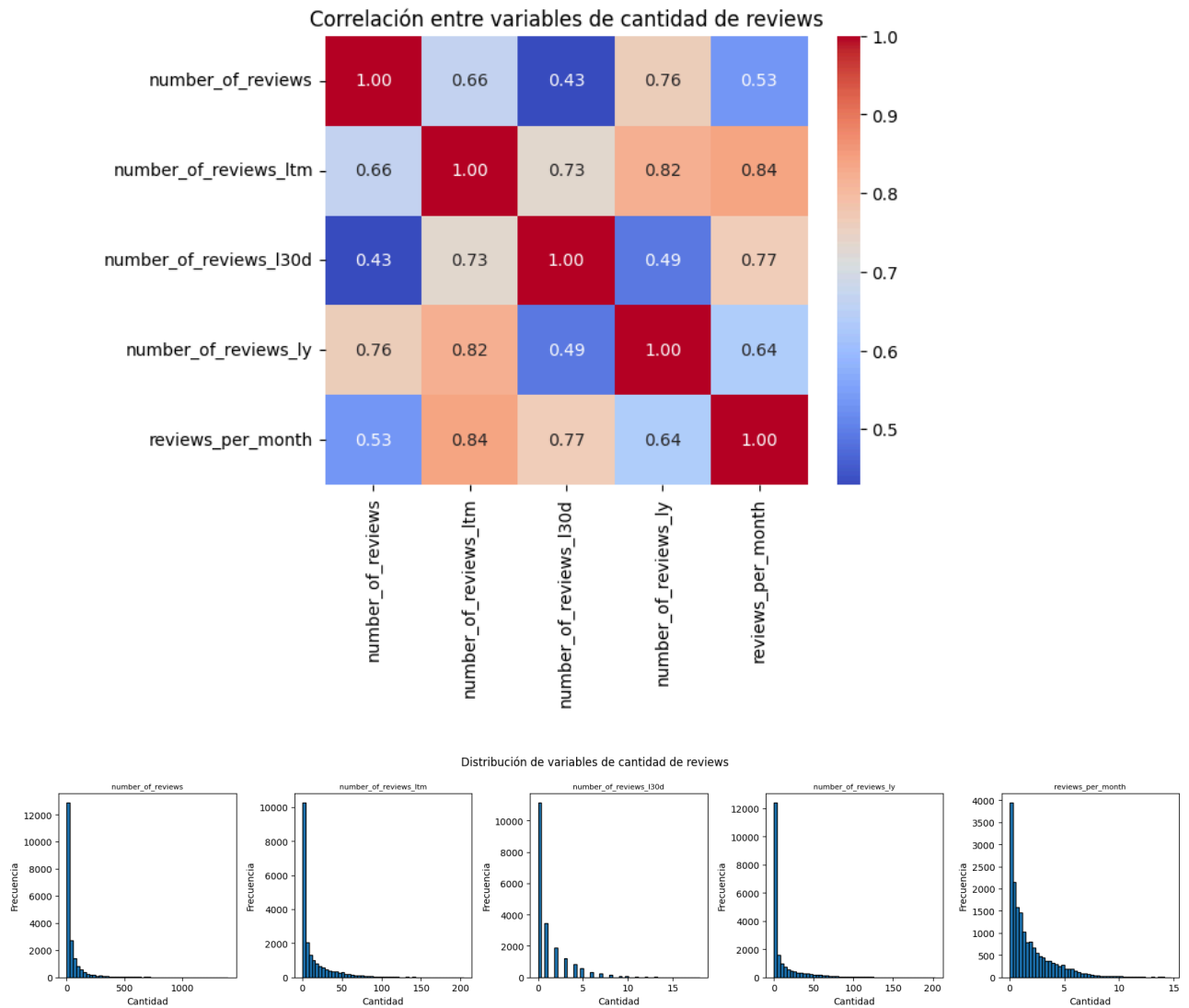
- **Variables redundantes de noches mínimas/máximas:** el heatmap de correlaciones reveló dos grupos de alta correlación interna (0.84–0.95 dentro de cada grupo). Se conservó un representante actual (`minimum_nights`, `maximum_nights`) y uno futuro (`minimum_nights_avg_ntm`, `maximum_nights_avg_ntm`) descartando al resto de variables de este grupo.



- La columna `license`, en lugar de descartarla completamente, se transformó en una variable binaria `has_license` y resultó más útil. Los valores nulos y las entradas Exempt o pendientes se interpretaron como ausencia de licencia.
- **Variables de cantidad de listings:** Se realizó un análisis similar, detectando que `calculated_host_listings_count_entire_homes` presenta correlación perfecta (1.00) con `calculated_host_listings_count`, y que `calculated_host_listings_count_shared_rooms` tiene varianza prácticamente nula. Ambas fueron descartadas, conservando `calculated_host_listings_count`, `host_total_listings_count` y `calculated_host_listings_count_private_rooms` por capturar cada una información geográficamente distinta.



- **Variables de cantidad de reviews:** Se analizó el heatmap con las correlaciones con el método de spearman y además estudiamos las distribuciones de cada una de estas variables. Decidimos eliminar `reviews_per_month` porque su información es calculable a partir de otras variables y por su cantidad de nulos. Además eliminamos `number_of_reviews_1y` por contener prácticamente la misma información que `number_of_reviews_ltm`.



- **Variables de review-scores:** Se realizó el mismo procedimiento para estudiar si existía redundancia pero decidimos conservarlas todas por el momento.

4. Clasificación de variables

Se realizó una reclasificación formal de las variables restantes:

Tipo	Variables
Cuantitativas continuas	price, latitude, longitude, host_response_rate, host_acceptance_rate, review_scores_*
Cuantitativas discretas	accommodates, bedrooms, beds, bathrooms, minimum_nights, number_of_reviews, etc.
Cualitativas ordinales	host_response_time

Cualitativas nominales	room_type, property_type, neighbourhood_cleansed
Booleanas	host_is_superhost, instant_bookable, has_license, etc.

Se corrigieron los tipos de datos de las variables booleanas (de strings "t"/"f" a bool), las de porcentaje (de strings "XX%" a float) y las de fechas (de strings a datetime). La variable `host_response_time` fue convertida a una Categorical ordenada.

5. Filtrado por reglas de negocio

Se aplicó un filtrado basado en **reglas de negocio** para las variables de capacidad del alojamiento, en lugar del Z-score modificado. La razón es que estas variables presentan distribuciones muy asimétricas concentradas en valores bajos: un apartamento con 3 baños es completamente válido pero el Z-score lo marcaría como outlier dado que la mayoría tiene

1. Los límites aplicados fueron: `accommodates ≤ 16`, `bedrooms ≤ 8`, `beds ≤ 12`, `bathrooms ≤ 6`, y `minimum_nights ≤ 200`.

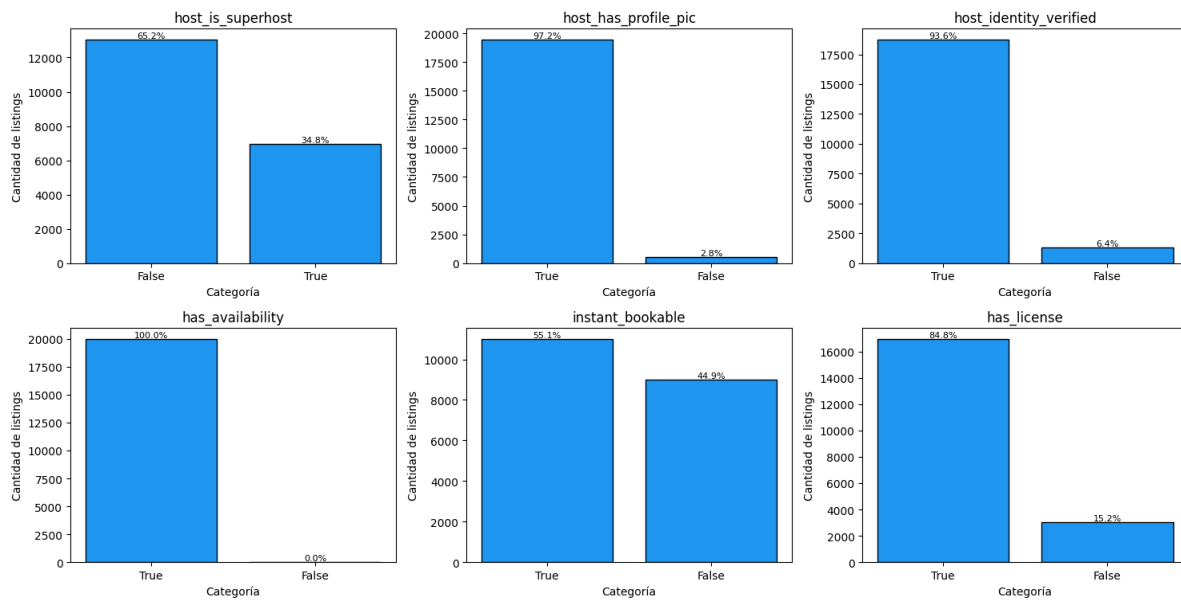
6. Análisis exploratorio

Univariado

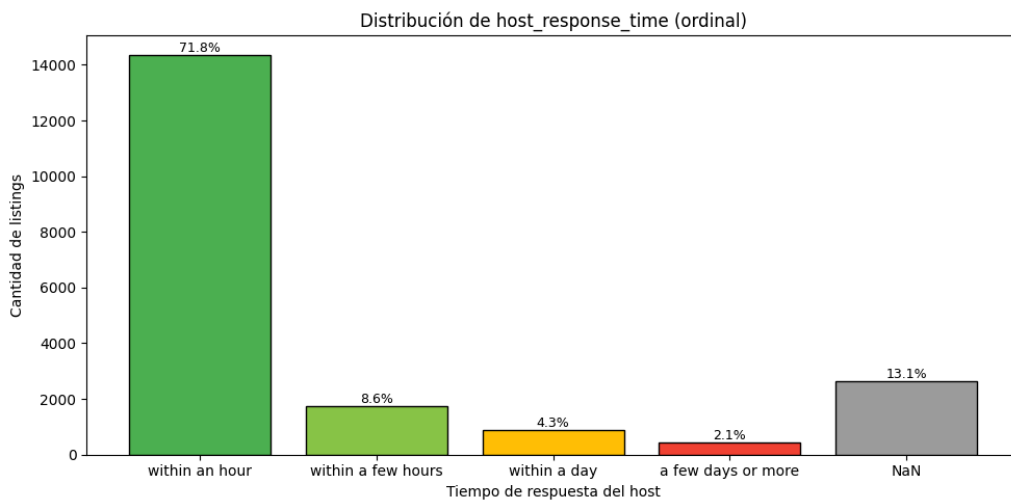
En el apartado de las variables numéricas, se mostró que las **variables de capacidad** (`accommodates`, `bedrooms`, `bathrooms`, `beds`) presentan distribuciones muy asimétricas concentradas en valores bajos, lo esperable en un mercado de departamentos pequeños. Las **variables de disponibilidad** (`availability_30`, `availability_365`) muestran distribuciones uniformes: `availability_365` tiene un pico marcado en 365 días, correspondiente a listings que están disponibles siempre. Los **review scores** están fuertemente concentrados entre 4.5 y 5.0, con pocos valores bajos, reflejando el sesgo de supervivencia (los listings con malas reseñas tienden a darse de baja). Las **variables de cantidad de reviews** presentan distribuciones muy asimétricas, concentrando la mayoría de las publicaciones con pocos reviews pero con algunos outliers de hasta 1400 reviews históricas. La situación de las **variables de listings del host** es muy similar a la del anterior grupo.

Continuando con las **variables categóricas**: Las booleanas presentaron distribuciones relativamente balanceadas y con posible valor predictivo, pero `host_has_profile_pic`, `host_identity_verified` y `has_availability` contenían casi todos sus valores en una única categoría, por lo que fueron eliminadas.

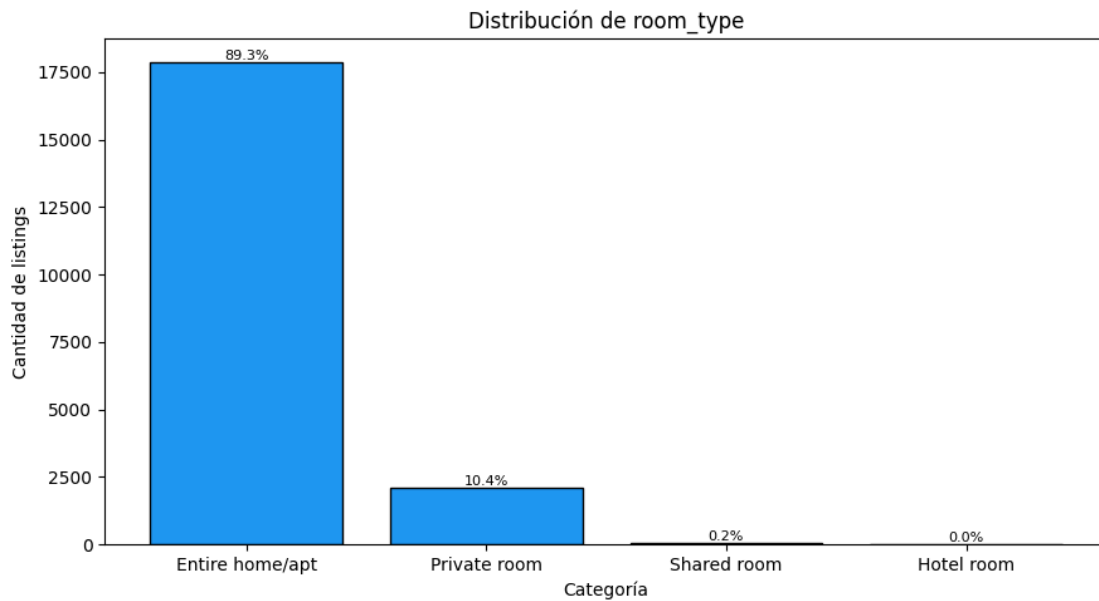
Distribución de variables booleanas



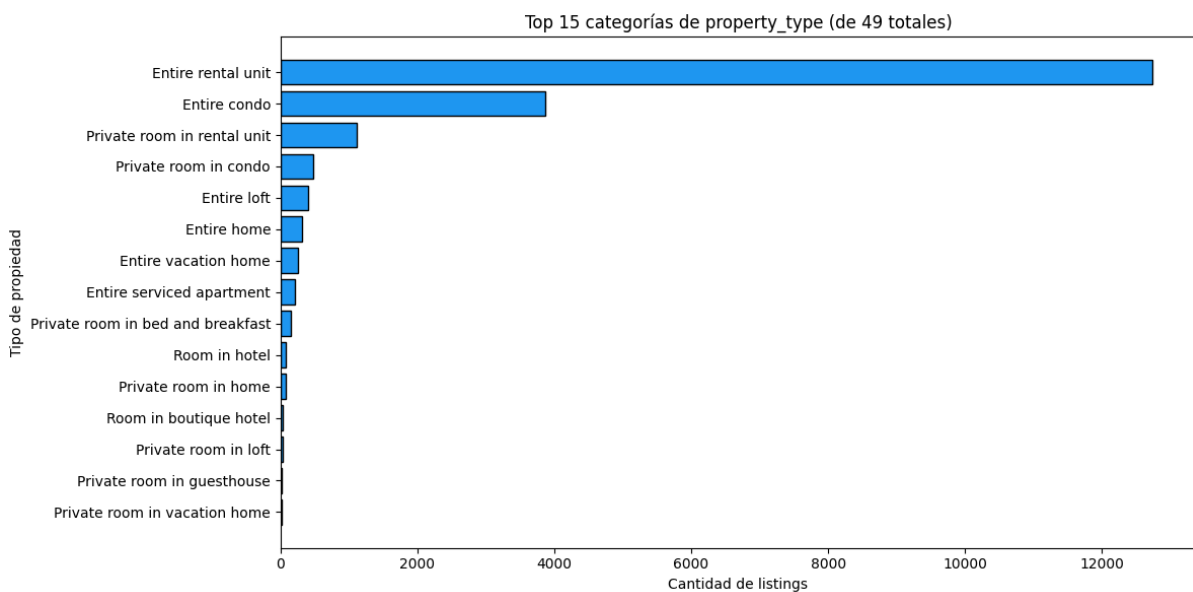
Se analizó la variable ordinal **host_response_time**, que refleja un mercado donde la mayoría de los hosts en Milán responden rápido debido a la alta competitividad en el mercado. Los casos nulos corresponden a hosts nuevos o con muy baja actividad que no cuentan con la suficiente información para categorizarlos.



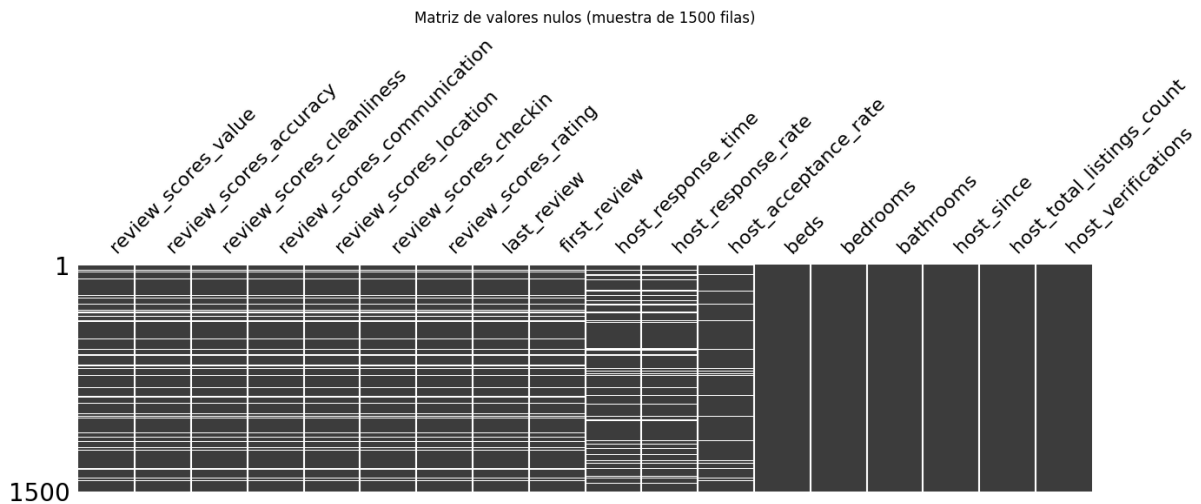
El estudio de **room_type** confirmó que los AirBnB en Milán son esencialmente departamentos enteros, con muy pocas habitaciones privadas y casi nada de compartidas ni habitaciones de hotel.



En `property_type` notamos que entre las 5 categorías se acumulaban más del 90% de los listings, decidiendo tratarla luego en la sección de feature engineering

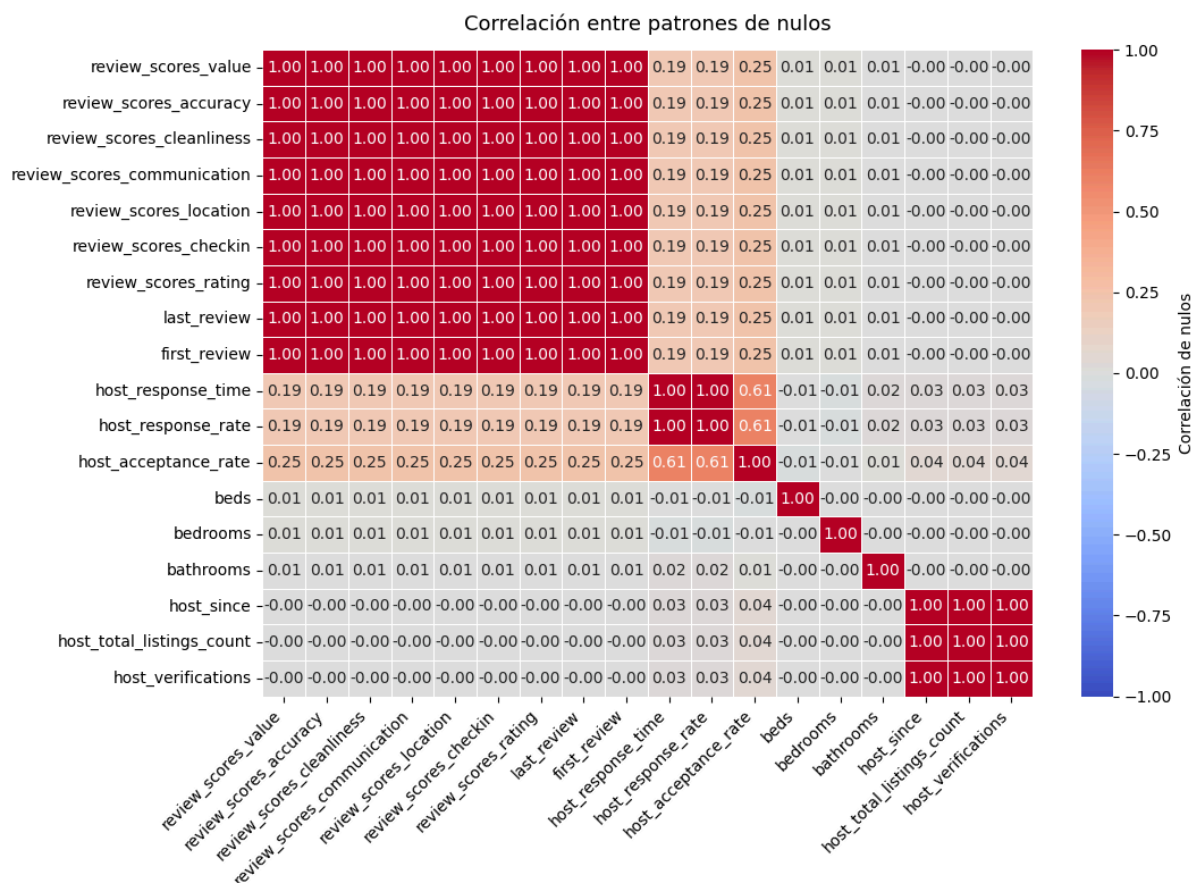


Para profundizar más en el análisis optamos verificamos si la cantidad de nulos de estas variables previamente mencionadas aparecen de forma aleatoria o siguen patrones estructurales. Para esto armamos una matriz visual de nulos donde gráficamente se vean en qué filas se presentan los nulos de las variables:



Posteriormente complementamos el análisis con heatmap de correlación de nulos donde:

- Valores cercanos a **1**: las dos variables tienden a estar nulas en las mismas filas
- Valores cercanos a **-1**: cuando una está nula, la otra tiende a estar presente
- Valores cercanos a **0**: nulos independientes entre sí

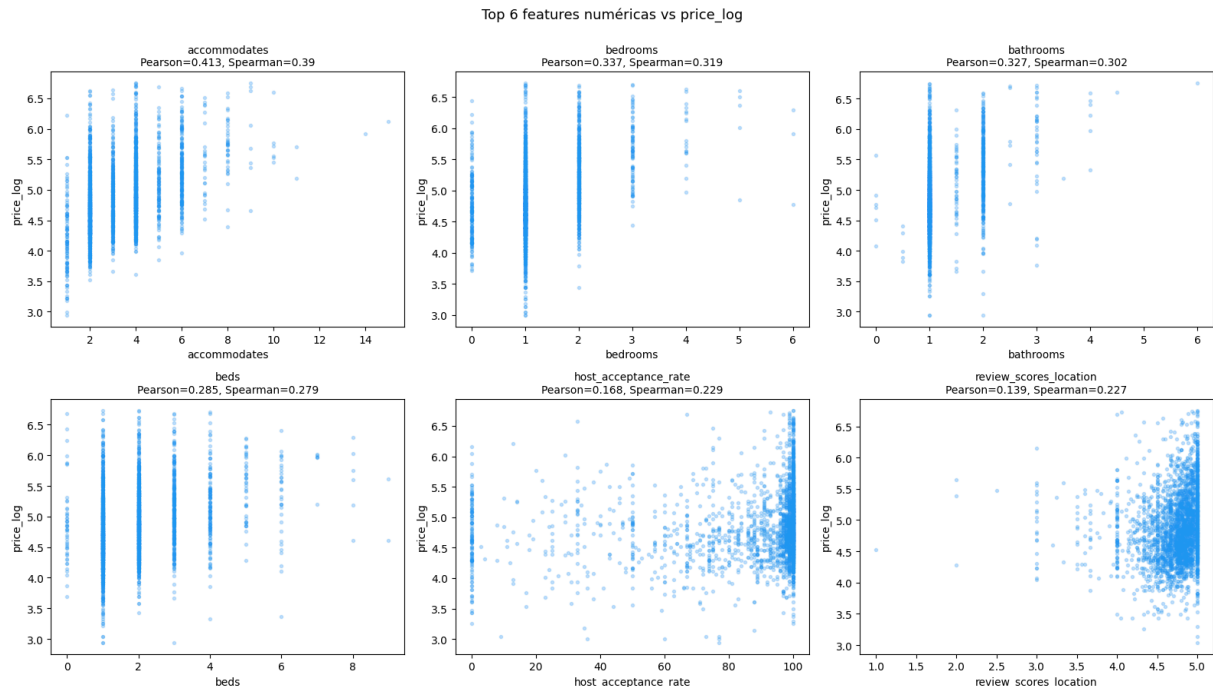


Desde este estudio se obtuvieron las siguientes conclusiones:

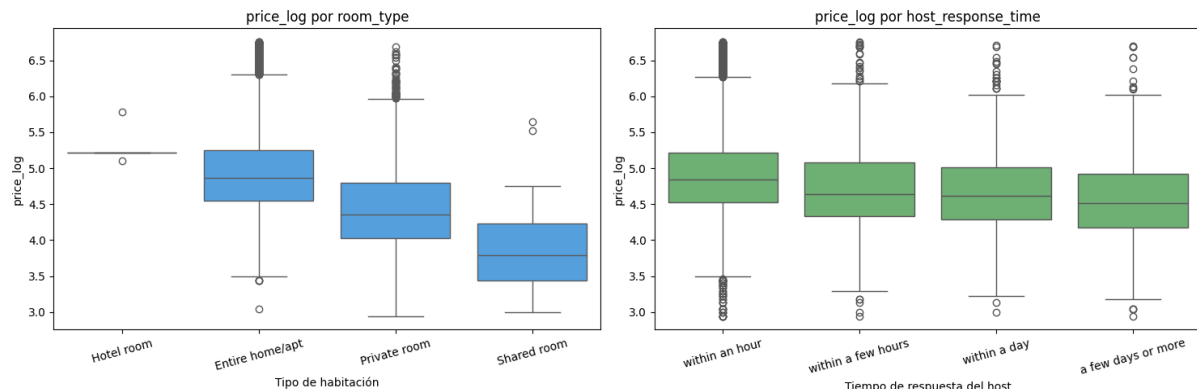
1. Los listings sin reviews tienen nulos en todas las variables relacionadas con reviews
2. Los hosts sin actividad de respuesta tienen nulos en `host_response_time` y `host_response_rate` juntos.

Bivariado

El **análisis bivariado** respecto a **price_log** reveló que las variables de capacidad son las que mayor correlación presentan con el target (Spearman entre 0.28 y 0.39), siendo **accommodates** la más correlacionada (Pearson=0.41).

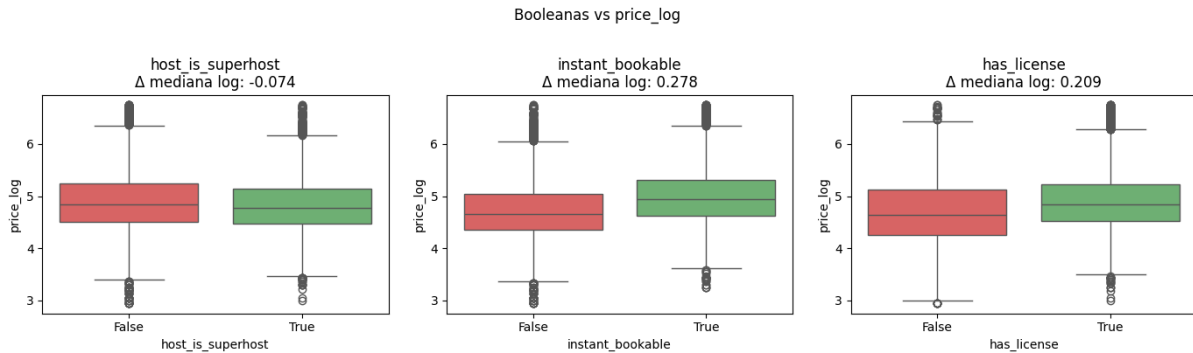


Entre las variables categóricas, **room_type** y **host_response_time** mostraron cómo afectan al precio final, siendo la primera una tendencia mucho más marcada

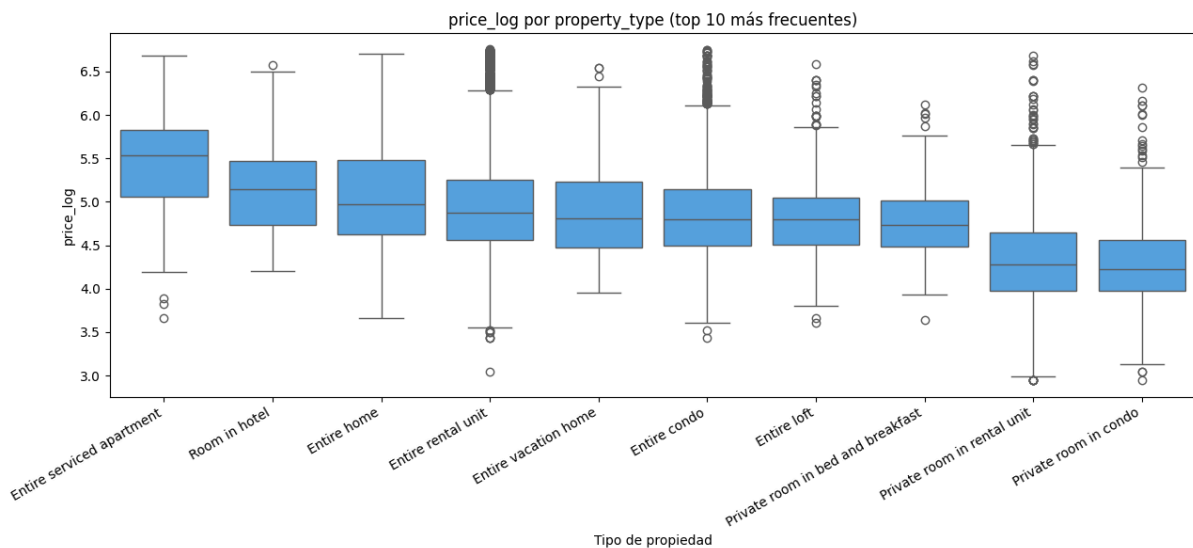


Las variables booleanas mostraron algunos resultados interesantes:

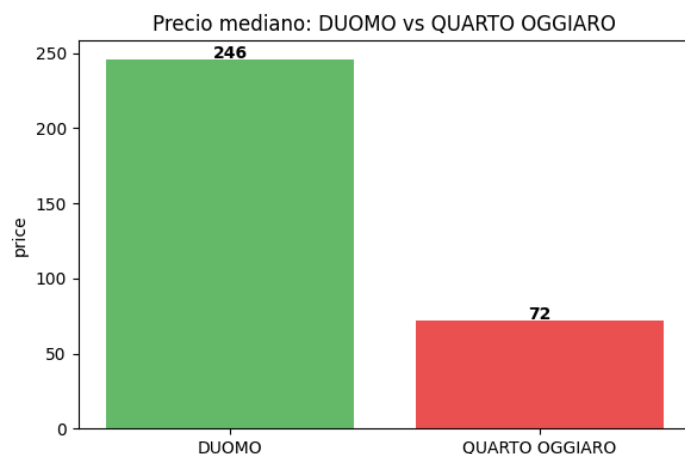
- **host_is_superhost** tuvo un resultado poco esperado, con los super hosts teniendo una mediana de precio menor. Una posible explicación es que estos priorizan aumentar los clientes por sobre el precio para acumular buenas reseñas.
- **instant_bookable** y **has_license** mostraron unas diferencias positivas por sobre el precio.



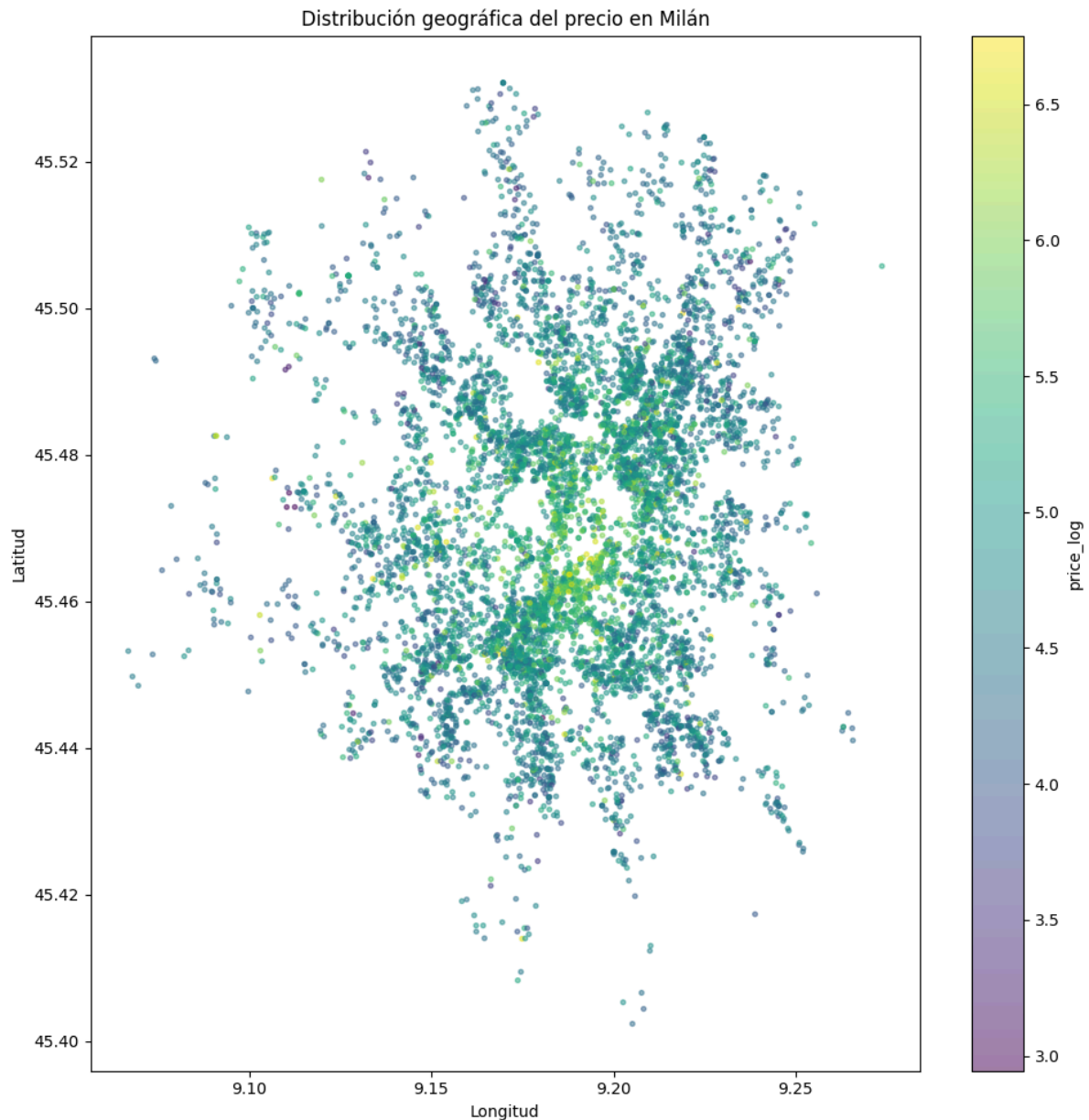
Se estudiaron las 10 categorías con más frecuencia de **property_type**, mostrando grandes diferencias de precio entre sus categorías. Vale la pena mencionar que las categorías **Entire *** en general cobran más que las **Private room ***, información que es consistente con lo observado en **room_type**.



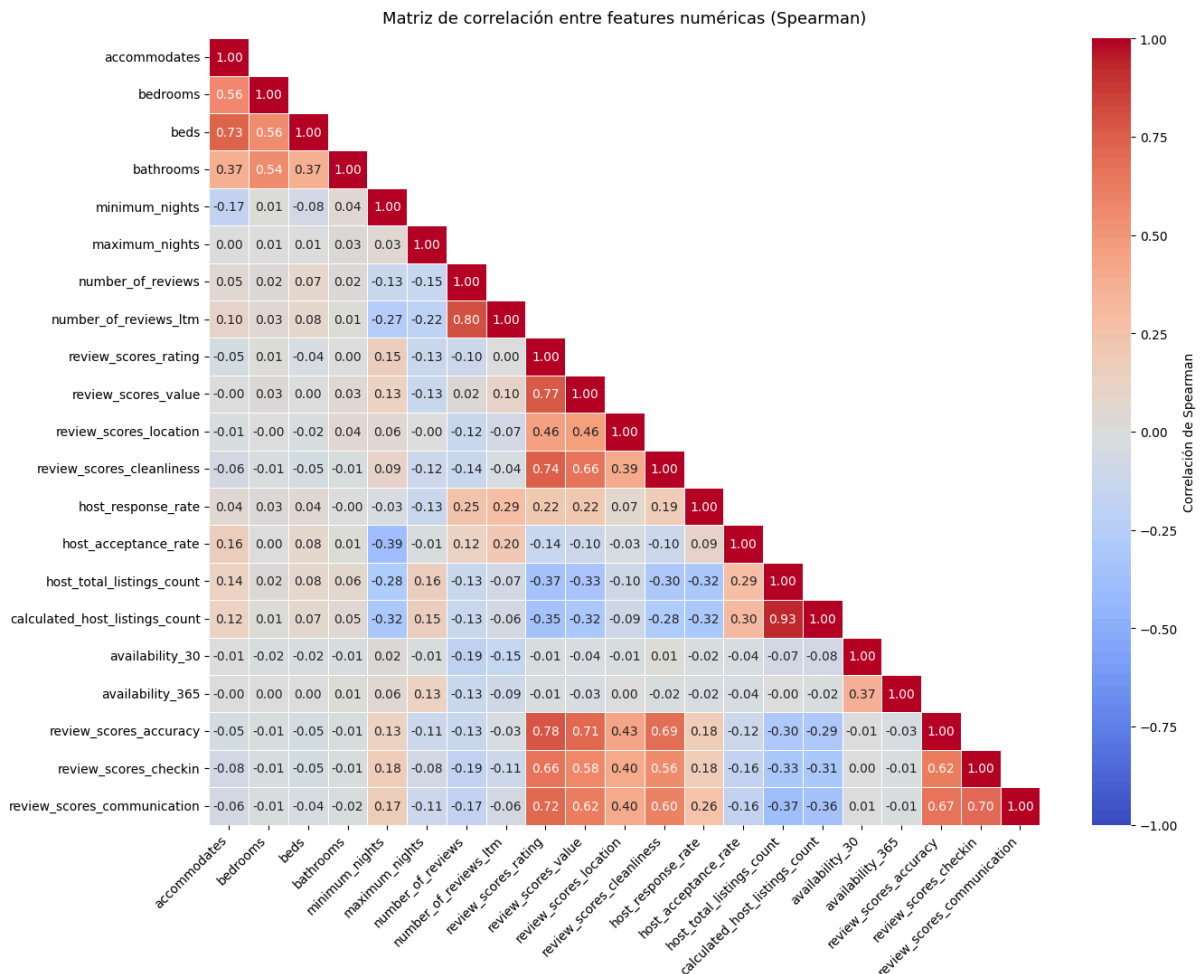
neighbourhood_cleansed mostró las diferencias más marcadas: los barrios **DUOMO** y **BRERA** tienen una mediana de precio más del triple que los barrios periféricos como **QUARTO OGGIARO**. La distribución geográfica del precio confirma visualmente esta concentración de precios altos en el centro de Milán.



Un mapa de calor organizado por latitud y longitud confirma que los precios más elevados se concentran en el centro de Milán, alrededor de (9.18, 45.46), que corresponde a la zona del DUOMO y BRERA. A medida que nos alejamos del centro hacia la periferia, los precios caen.



La **matriz de correlación entre features** (Spearman) detectó los siguientes pares problemáticos de multicolinealidad, que llevaron a eliminar las variables mencionadas:



- `host_total_listings_count` vs `calculated_host_listings_count`: correlación 0.9+, se descartó la primera.
- `number_of_reviews` vs `number_of_reviews_ltm`: correlación 0.80, se descartó la primera ya que la actividad reciente es más relevante que el historial total.
- Los seis sub-puntajes de reseñas (`accuracy`, `cleanliness`, `checkin`, `communication`, `value`) presentaron correlaciones entre 0.66 y 0.78 con `review_scores_rating`. Se conservaron únicamente `review_scores_rating` y `review_scores_location`, ya que esta última mostró ser la más independiente del resto (correlación 0.44–0.46 con los demás scores), capturando información geográfica objetiva que el puntaje global no recoge.

7. División train/test

La división en conjunto de entrenamiento (80%) y de prueba (20%) se realizó **antes** del Feature Engineering y la imputación de valores nulos. Esta decisión es crucial para evitar data leakage: cualquier estadístico calculado para imputar nulos (medianas, modas) debe aprenderse únicamente del conjunto de entrenamiento para garantizar que el conjunto de prueba simula datos realmente nuevos.

8. Feature Engineering

Se generaron las siguientes variables derivadas:

- **has_reviews** e **is_active**: flags binarias que capturan el ciclo de vida del listing. La primera indica si el alojamiento recibió alguna reseña en su historia; la segunda si recibió reseñas en los últimos 12 meses. La combinación define tres estados: *nuevo*, *dormant* (tuvo reseñas pero ya no) y *activo*.
- **has_response_data**: indica si el host tiene historial registrado de respuesta a solicitudes. Permite interpretar correctamente los nulos de **host_response_time** y **host_response_rate**.
- **Variables temporales** (**host_age_days**, **listing_age_days**, **days_since_last_review**): calculadas como diferencia en días respecto a la fecha del scrape (22/09/2025), transformando las fechas en medidas numéricas.
- Corrección de valores temporales negativos: detectamos casos con **listing_age_days** y **days_since_last_review** ligeramente negativos (min -1 y -2 días). Esto se debe a discrepancias de zona horaria entre el momento del scrape y las reviews registradas en hora local de Milán. Truncamos estos valores a 0 ya que son semánticamente equivalentes a "review del día del scrape".
- **n_amenities**: conteo de comodidades a partir del parsing de la lista en la columna **amenities**, que sirve como indicio del nivel de equipamiento del alojamiento.
- **n_verifications** fue analizada y descartada por presentar varianza casi nula: el 95.4% de los listings tenía exactamente 2 o 3 verificaciones.

Para el **encoding de variables categóricas** se tomaron las siguientes decisiones:

- **neighbourhood_cleansed** fue eliminada. Sus 88 barrios necesitan 87 columnas extra para OHE. Dado que **latitude** y **longitude** ya capturan la información geográfica de forma continua y sin ese costo dimensional, se consideró redundante.
- **property_type** tiene 50 categorías, de las cuales las 8 que superan el 1% de frecuencia acumulan el 97% de los listings. Se agruparon las categorías minoritarias en una categoría 'Other' y se aplicó OHE sobre 8 principales.
- **room_type** fue tratada con OHE estándar al tener solo 4 categorías (3 columnas).
- **host_response_time** fue codificada numéricamente respetando su orden implícito (0 = "within an hour", 3 = "a few days or more").

9. Tratamiento de nulos y escalado

Todos los nulos fueron imputados utilizando estadísticos calculados **exclusivamente sobre el conjunto de entrenamiento**, aplicando luego los mismos valores al conjunto de prueba. Se utilizó la **mediana** para variables numéricas (**bathrooms**, **bedrooms**, **beds**, **host_acceptance_rate**, **host_age_days**, **host_response_rate**) y la **moda** para **host_response_time**. Las variables de reseñas (**review_scores_rating**, **review_scores_location**, **listing_age_days**, **days_since_last_review**) se imputaron con 0 cuando el motivo del nulo era la ausencia de reseñas, información ya capturada en la variable **has_reviews**.

Se aplicó `StandardScaler` ajustado únicamente sobre el conjunto de entrenamiento, transformando luego ambos conjuntos con el mismo escalador. Esta transformación es necesaria para la Regresión Lineal. Para XGBoost y Random Forest no alteraron los resultados dado que los modelos basados en árboles son invariantes a la escala, pero se mantuvieron para unificar.

10. Entrenamiento y predicción

Los tres modelos fueron entrenados usando `price_log` como variable objetivo. Esta decisión es consistente en los tres casos: trabajar en escala logarítmica estabiliza la varianza, reduce el impacto de los precios extremos y mejora el ajuste de la Regresión Lineal. Las predicciones se revierten a escala USD mediante `expm1` para reportar métricas interpretables.

Modelo 1: Regresión Lineal Múltiple

La Regresión Lineal fue entrenada sobre el conjunto de entrenamiento escalado usando todas las features disponibles tras el preprocesamiento. No se realizó selección de features adicionales ya que el objetivo principal de este modelo es establecer una línea de base y analizar los coeficientes para identificar las variables más influyentes.

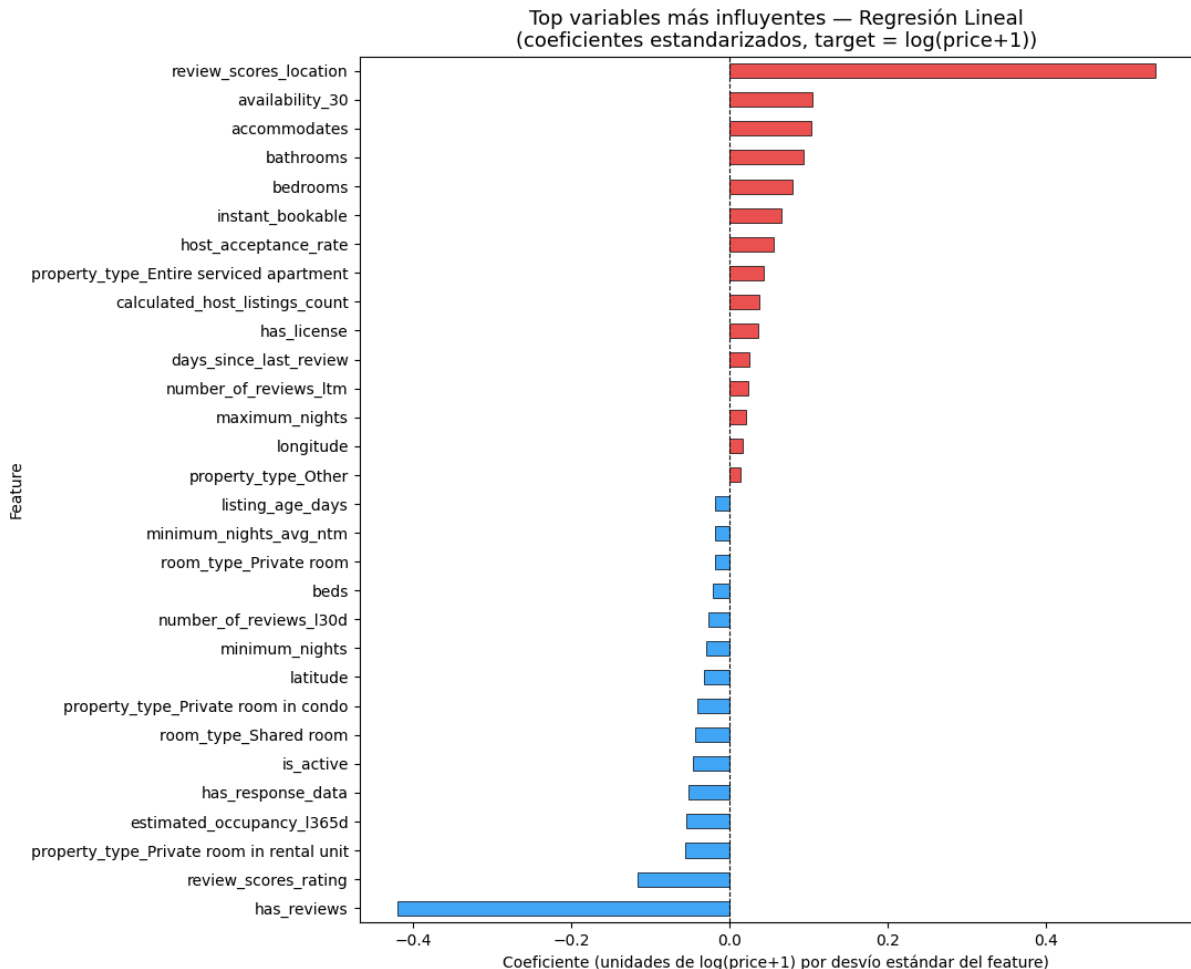
Resultados:

Métrica	Train	Test
R^2 (escala log)	0.3919	0.4144
R^2 (escala USD)	0.2627	0.3073
RMSE (\$)	\$93.35	\$90.61
MAE (\$)	\$54.95	\$53.51
MAPE	34.98%	33.83%

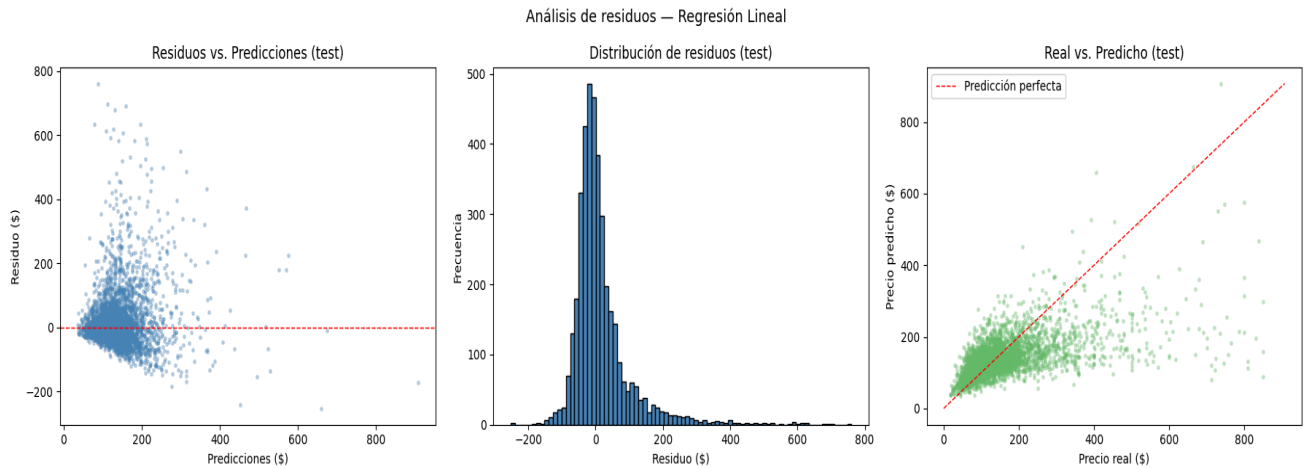
El resultado más llamativo es que **las métricas de test superan a las de train** en todas las escalas. Esto es consecuencia del bajo sesgo intrínseco del modelo: la Regresión Lineal no memoriza el conjunto de entrenamiento (baja varianza), y puede depender en cierta medida de la aleatoriedad del split. No indica overfitting en el sentido habitual ni tampoco que el modelo sea particularmente bueno: en ambos conjuntos el R^2 es inferior al 0.42, lo que significa que el modelo explica menos del 42% de la varianza del precio.

La limitación fundamental es estructural: la Regresión Lineal asume relaciones lineales entre cada feature y el target, manteniendo las demás constantes. Este supuesto no se cumple en el problema: el efecto de `bedrooms` sobre el precio varía según `room_type`, la geolocalización tiene efectos no lineales, y la interacción entre variables de capacidad es relevante. El MAE de ~\$54 sobre una mediana de precio de ~\$125 y un MAPE de ~34% son el resultado esperable para un modelo que intenta capturar con una recta un fenómeno que no es lineal.

Variables más influyentes: analizando los coeficientes estandarizados (directamente comparables entre sí), `review_scores_location` tiene el mayor coeficiente positivo: la percepción de ubicación por parte de los huéspedes es el predictor lineal más fuerte del precio, incluso por encima de la capacidad física del alojamiento. `has_reviews` tiene el coeficiente negativo más pronunciado, capturando que los listings sin ninguna reseña tienden a tener precios más altos en promedio, posiblemente porque corresponden a propiedades nuevas o de lujo que aún no han acumulado historial. Las variables de capacidad (`accommodates`, `bathrooms`, `bedrooms`) también presentan coeficientes positivos relevantes y consistentes con el análisis exploratorio.



El análisis de residuos confirmó los límites del modelo: se observa un patrón de abanico en el gráfico de residuos vs. predicciones, con varianza creciente para precios predichos más altos. La distribución de residuos muestra sesgo a la derecha, indicando que el modelo subestima con mayor frecuencia los precios altos. El gráfico real vs. predicho muestra que el modelo captura razonablemente el rango bajo-medio (hasta ~\$300), pero pierde capacidad predictiva para los alojamientos más caros.



Modelo 2: XGBoost

XGBoost es un algoritmo de gradient boosting que construye árboles de decisión de forma **secuencial**: cada árbol nuevo aprende a corregir los errores residuales del ensamble anterior. Esta propiedad le permite capturar relaciones no lineales e interacciones entre variables que la Regresión Lineal no puede representar, a costa de un mayor riesgo de overfitting si no se regulariza adecuadamente.

Se entrenó primero un modelo base con parámetros por defecto para establecer un punto de referencia: en esa configuración el modelo alcanzó R^2 log train=0.9623, lo que confirmó la necesidad de aplicar regularización con el objetivo de controlar el overfitting. La optimización de hiperparámetros se realizó mediante **RandomizedSearchCV** con `KFold(n_splits=5)`, evaluando 60 combinaciones aleatorias del espacio de búsqueda con RMSE en escala log como métrica de optimización. Se optó por RandomizedSearchCV sobre GridSearchCV dado que el espacio de búsqueda era amplio (combinaciones de nueve hiperparámetros), lo que hace al barrido exhaustivo computacionalmente prohibitivo.

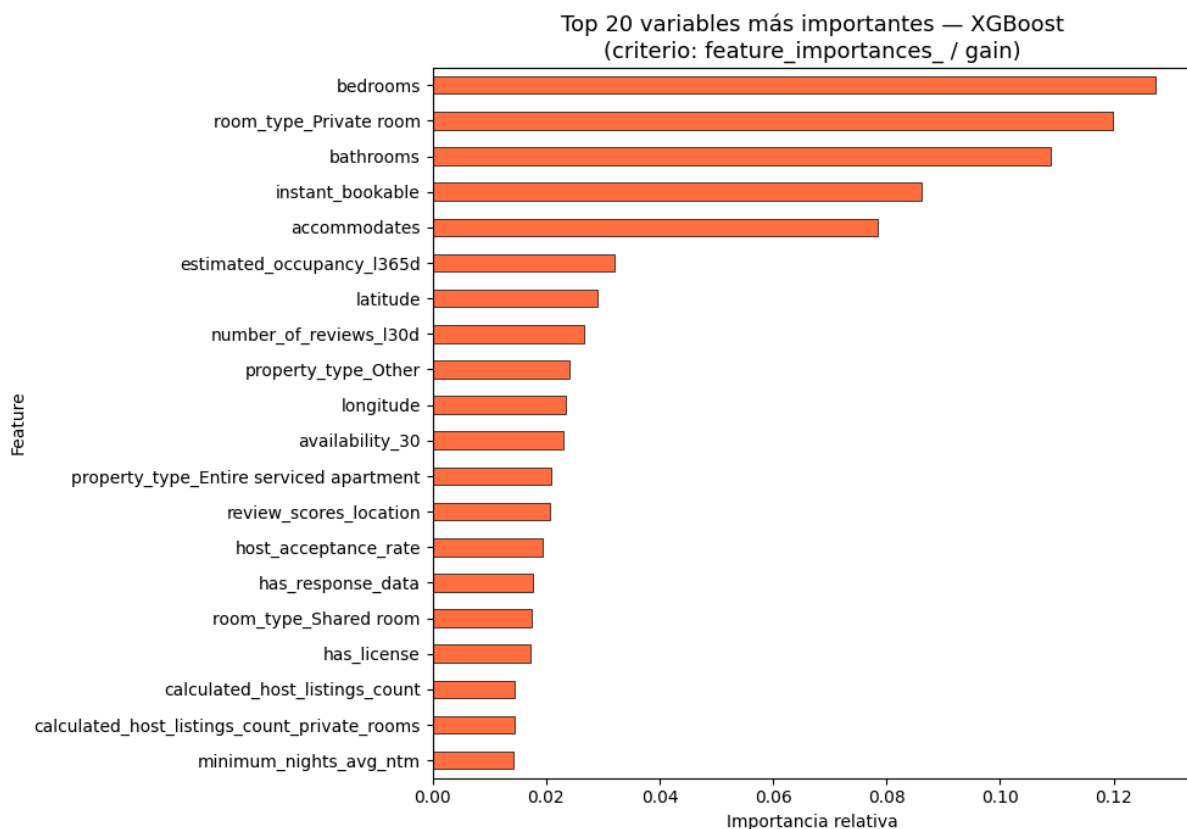
Los hiperparámetros explorados y su rol en el modelo son: `n_estimators` (cantidad de árboles en el ensamble), `max_depth` (profundidad máxima, controla complejidad), `learning_rate` (shrinkage de la contribución de cada árbol; valores bajos requieren más árboles pero generalizan mejor), `min_child_weight` (mínimo de observaciones por hoja, equivalente al `min_samples_leaf` de Random Forest), `gamma` (penalización por split, reduce la cantidad de divisiones no productivas), `subsample` y `colsample_bytree` (muestreo de filas y features por árbol, respectivamente), `reg_alpha` y `reg_lambda` (regularización L1 y L2 sobre los pesos de las hojas).

Resultados (modelo optimizado):

Métrica	Train	Test
R ² (escala log)	0.7709	0.6407
R ² (escala USD)	0.6574	0.5235
RMSE (\$)	\$63.63	\$75.15
MAE (\$)	\$33.48	\$40.23
MAPE	19.93%	24.85%

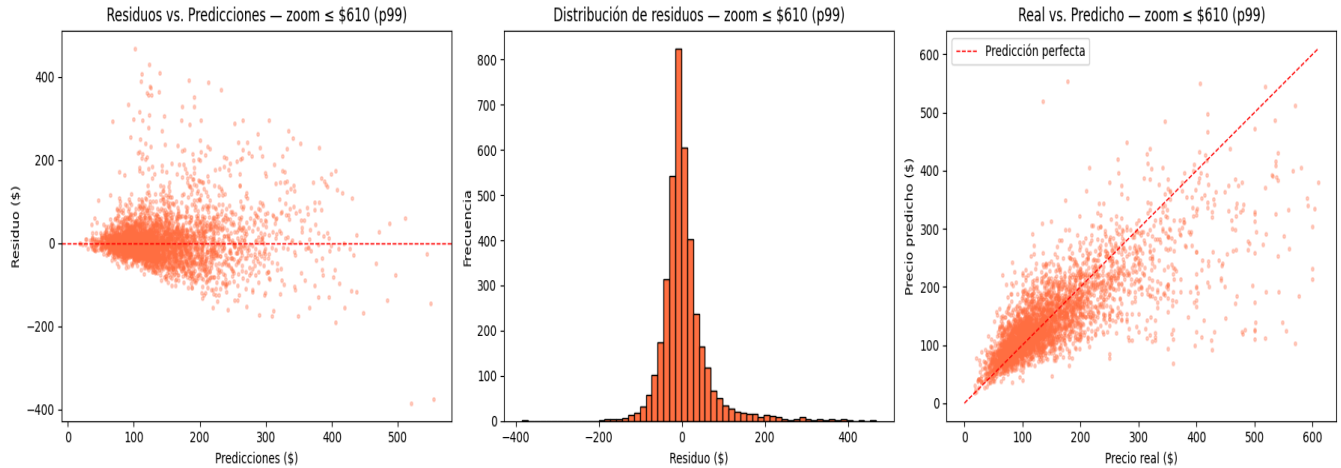
El gap train-test en R² log es de 0.130, el menor de los tres modelos, lo que indica que la regularización fue efectiva: los parámetros encontrados (entre ellos `reg_lambda` y `min_child_weight` elevados) redujeron el R² train de 0.962 del modelo base a 0.771, priorizando la generalización.

Variables más importantes (criterio gain): el tipo de habitación (`room_type_Private room`) y la capacidad física (`bedrooms`, `bathrooms`, `accommodates`) dominan la importancia, seguidos por condiciones de reserva (`instant_bookable`). La geolocalización (`latitude`, `longitude`) aparece en posiciones medias, reflejando que en Milán la ubicación importa pero el tipo y tamaño del alojamiento son los determinantes primarios.



El análisis de residuos mostró mejoras cualitativas respecto a la Regresión Lineal: la distribución de residuos es más simétrica, y el gráfico real vs. predicho muestra mayor adhesión a la diagonal, incluso para precios intermedios.

Análisis de residuos — XGBoost (zoom al 99% del test, n=3958 obs.)



Modelo 3: Random Forest

Se eligió **Random Forest** como tercer modelo por su valor comparativo respecto a XGBoost. Mientras XGBoost construye árboles de forma secuencial (*boosting*), Random Forest los construye de forma paralela e independiente promediando sus predicciones (*bagging*). Esta diferencia permite evaluar si el problema se beneficia más de la corrección iterativa de errores o de la reducción de varianza por promedio, que en teoría hace a Random Forest menos susceptible al overfitting.

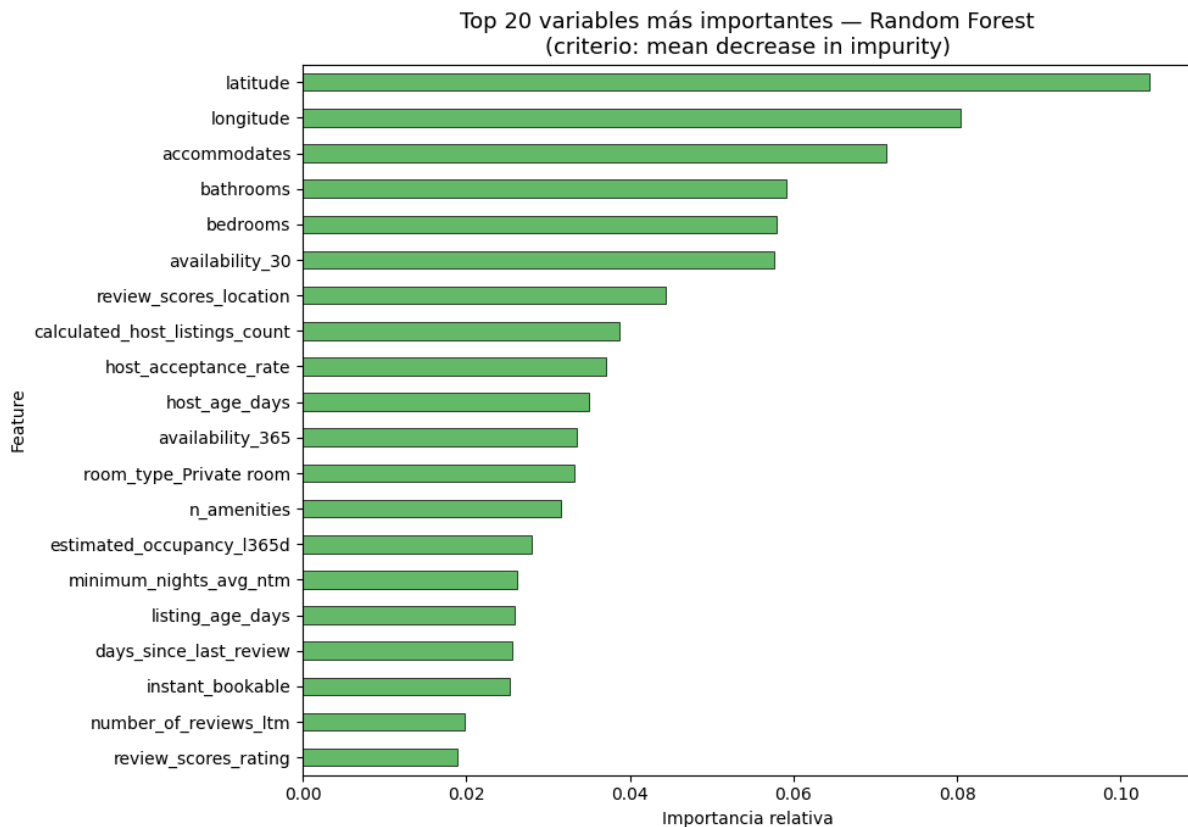
Al igual que XGBoost, se realizó primero un entrenamiento base que confirmó overfitting severo (R^2 train=0.9440). La optimización fue con **RandomizedSearchCV** y `KFold(n_splits=5)`, evaluando 40 combinaciones. El mismo esquema de validación cruzada que en XGBoost permite comparar los resultados de CV entre ambos modelos de forma consistente. Se exploró el espacio de `n_estimators`, `max_depth`, `max_features`, `min_samples_split`, `min_samples_leaf` y `bootstrap`.

Los mejores hiperparámetros encontrados fueron: `n_estimators=300`, `max_depth=20`, `max_features=0.3`, `min_samples_leaf=2`, `min_samples_split=5`, `bootstrap=True`.

Resultados (modelo optimizado):

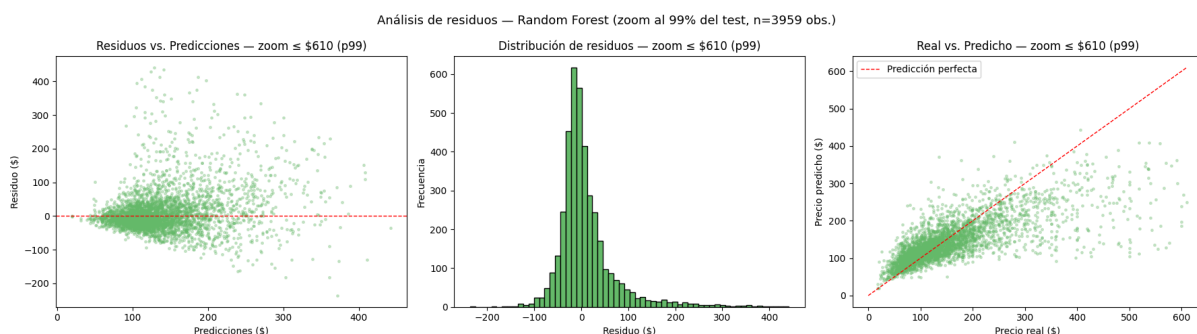
Métrica	Train	Test
R^2 (escala log)	0.8870	0.5986
R^2 (escala USD)	0.7710	0.4492
RMSE (\$)	\$52.03	\$80.80
MAE (\$)	\$24.23	\$43.22
MAPE	13.57%	26.65%

Variables más importantes (criterio: mean decrease in impurity): a diferencia de XGBoost, en Random Forest la geolocalización lidera el ranking: **latitude** es la variable más importante con una importancia relativa de ~ 0.103 , seguida por **longitude** (~ 0.081). Recién en tercer lugar aparece **accommodates** (~ 0.071), seguido por **bathrooms**, **bedrooms** y **availability_30** con valores similares (~ 0.06). Este resultado contrasta con XGBoost, donde **bedrooms** encabezaba el ranking: en Random Forest aparece recién en la posición 5.



Análisis de residuos: el gráfico de residuos vs. predicciones muestra un patrón de abanico pronunciado, con varianza creciente a medida que aumenta el precio predicho.

Notablemente, para predicciones bajas (por debajo de $\sim \$100$) los residuos son en su mayoría positivos, indicando que el modelo subestima los alojamientos de precio bajo. La distribución de residuos es aproximadamente simétrica y fuertemente concentrada en torno al cero. El gráfico real vs. predicho muestra buena adhesión a la diagonal en el rango bajo-medio, pero con dispersión para precios reales superiores a $\sim \$300$, donde el modelo pierde capacidad predictiva de forma similar a XGBoost.



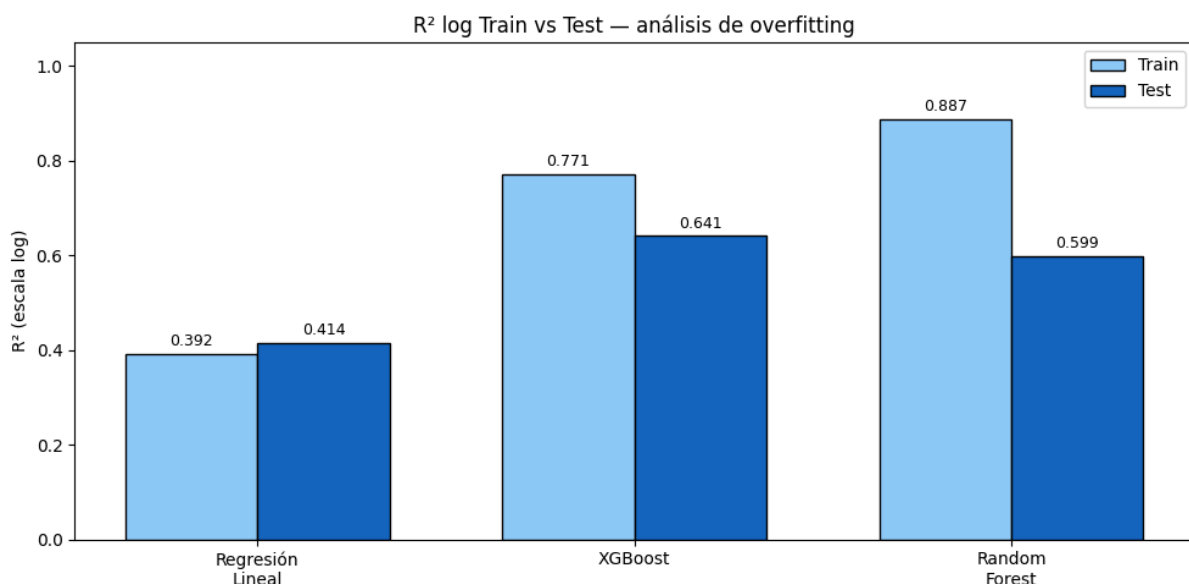
11. Elección del modelo

Modelo elegido: XGBoost

La elección se basa en el análisis conjunto de tres criterios: performance en test, control del overfitting y consistencia entre métricas.

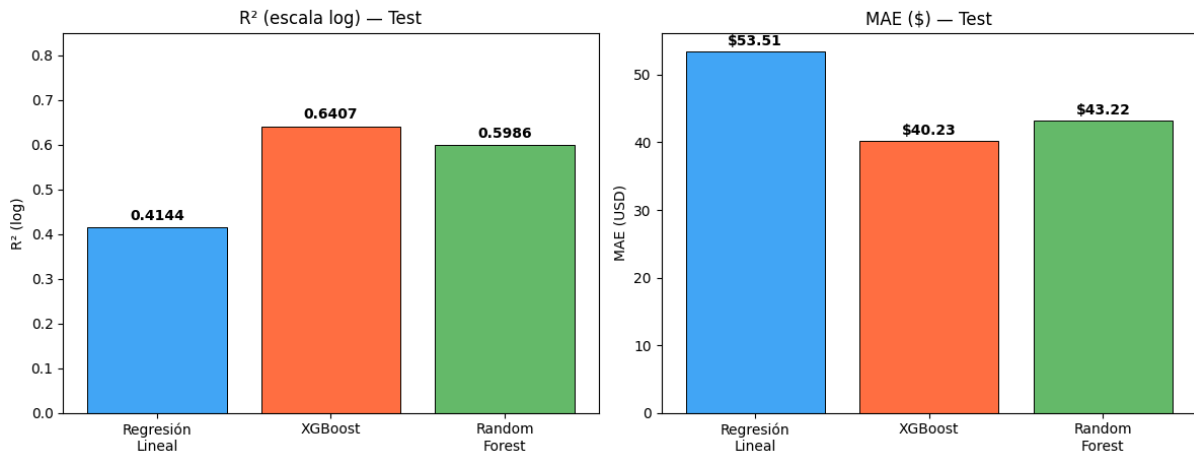
En **performance en test**, XGBoost obtiene el mejor R^2 log (0.641), el menor MAE (\$40.23) y el menor MAPE (24.85%) entre los tres modelos.

En **control del overfitting**, el gap train-test de XGBoost en R^2 log es de 0.130, el menor de los tres. Random Forest presenta un gap de 0.288, casi el doble, indicando mayor memorización del conjunto de entrenamiento. La Regresión Lineal muestra el fenómeno inverso (gap negativo: test ligeramente mejor que train), consecuencia de su bajo sesgo y la aleatoriedad del split, pero su performance absoluta es la peor. La optimización de hiperparámetros fue determinante en el caso de XGBoost: los parámetros de regularización redujeron el R^2 train de 0.962 (modelo base) a 0.771, priorizando la generalización y asegurando que el modelo aprende patrones del mercado en lugar de memorizar el conjunto de entrenamiento.



En **consistencia entre métricas**, XGBoost lidera simultáneamente en R^2 log, R^2 USD, MAE, RMSE y MAPE en el conjunto de prueba. Random Forest es competitivo pero no supera a XGBoost en ninguna métrica de test. La Regresión Lineal, como era esperable dado el carácter no lineal del problema y las interacciones entre variables de capacidad, tipo de habitación y ubicación, no logra capturar de manera eficiente la complejidad del mercado.

Comparación de modelos — métricas en Test



Cuadro comparativo de resultados

Modelo	R² log Train	R² log Test	Gap R² log	R² USD Test	RMSE (\$) Test	MAE (\$) Test	MAPE Test
Regresión Lineal	0.3919	0.4144	-0.0225	0.3073	\$90.61	\$53.51	33.83%
XGBoost	0.7709	0.6407	0.1302	0.5235	\$75.15	\$40.23	24.85%
Random Forest	0.8870	0.5986	0.2884	0.4492	\$80.80	\$43.22	26.65%

Por estas razones, **XGBoost es el modelo recomendado para predecir el precio de un alojamiento de Airbnb en Milán.**

EJ4: Clustering

1. Breve descripción y análisis del dataset

Los datos contienen información sobre 750 canciones de Spotify, con características de audio extraídas a través de la Spotify Web API. El archivo está en formato `.csv` y se carga usando el método específico para ello de pandas.

Cada fila representa una canción e incluye variables que describen propiedades acústicas y musicales como energía, danzabilidad, acústica, entre otras.

Se excluyeron del clustering las variables `key` y `mode` por las siguientes razones:

- `key` representa la tonalidad de la canción con valores enteros del 0 al 11, donde cada número corresponde a una nota musical. Sin embargo, la distancia numérica entre estos valores no tiene significado real: la diferencia entre Do (0) y Si (11) no implica que sean tonalidades más distantes que Do (0) y Re (2).
- `mode` es una variable nominal binaria (0=menor, 1=mayor) sin orden intrínseco.

En ambos casos, K-Means interpretaría la distancia entre sus valores como una diferencia métrica real, introduciendo ruido en el cálculo de distancias euclidianas sin aportar información musicalmente válida. Por estas razones se trabajó únicamente con las 11 variables restantes.

No se detectaron valores nulos, por lo que no fue necesario realizar imputaciones. En cambio se detectaron outliers principalmente en `instrumentalness` (21.6%), `speechiness` (12.8%) y `duration` (7.1%), entre otras.

Decidimos mantenerlos ya que no necesariamente son errores, sino que puede tratarse de canciones con características más peculiares, por ejemplo una canción que sea completamente instrumental o canciones muy largas.

Adicionalmente, como se trata de un problema de clustering los outliers permiten definir los grupos más diferenciados.

Lo que sí haremos para mitigar la influencia que podrían tener sobre K-Means es el escalado.

2. Escalado

K-Means asigna cada punto al centroide más cercano usando distancia euclidiana, por lo que es sensible a la magnitud de las variables. En este dataset conviven variables acotadas entre 0 y 1 (como `energy` o `valence`) con otras de rangos mucho mayores como `duration` (del orden de los 200.000 ms) o `loudness` (-60 a 0 dB). Sin escalar, estas últimas dominan el cálculo de distancias y harían irrelevantes al resto.

Se aplica `StandardScaler`, que transforma cada variable a media 0 y desvío estándar 1, garantizando que todas contribuyan por igual.

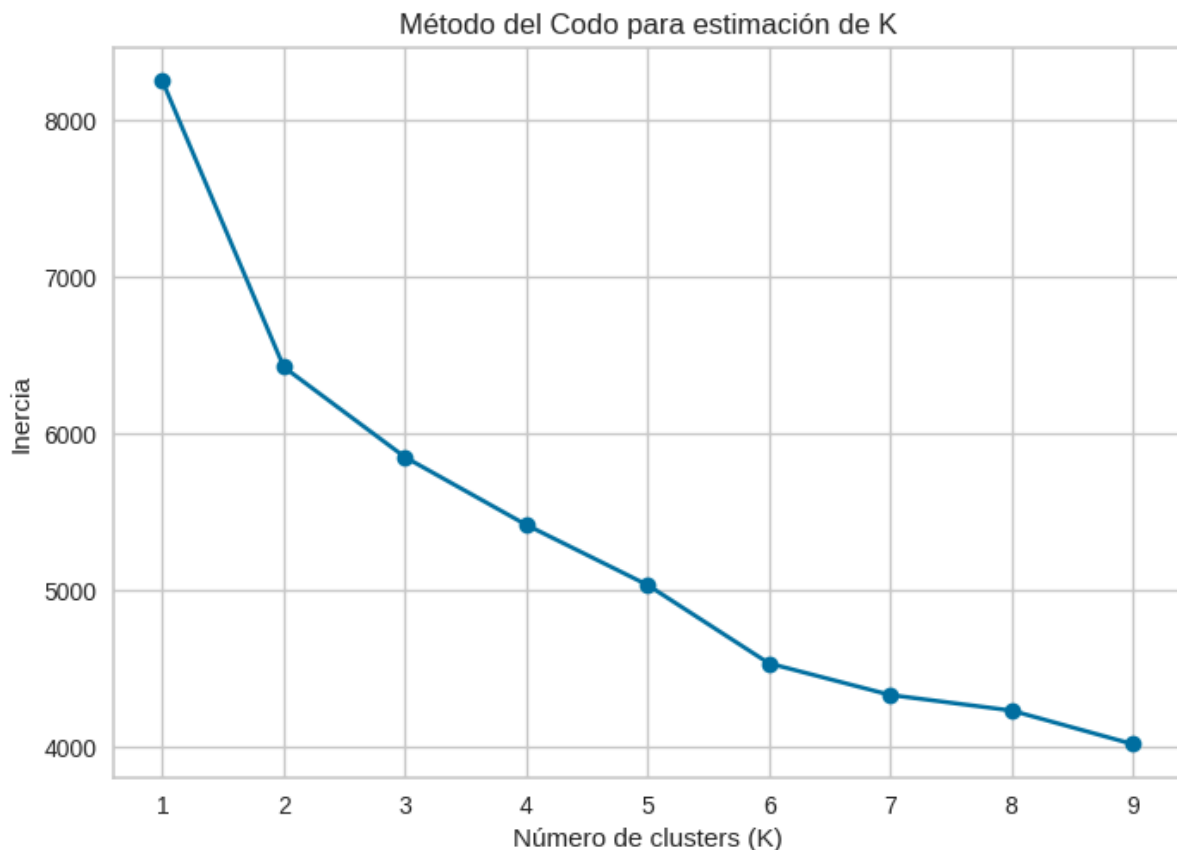
3. Tendencia al clustering

Para evaluar si el dataset presenta una estructura de agrupación natural se calculó el **Índice de Hopkins**, promediando 15 ejecuciones para obtener un valor más representativo. El resultado obtenido fue $H \approx 0.82$, que al ser mayor a 0.75 indica una fuerte tendencia al clustering. Esto confirma que los datos no están distribuidos aleatoriamente y que la aplicación de K-Means está justificada.

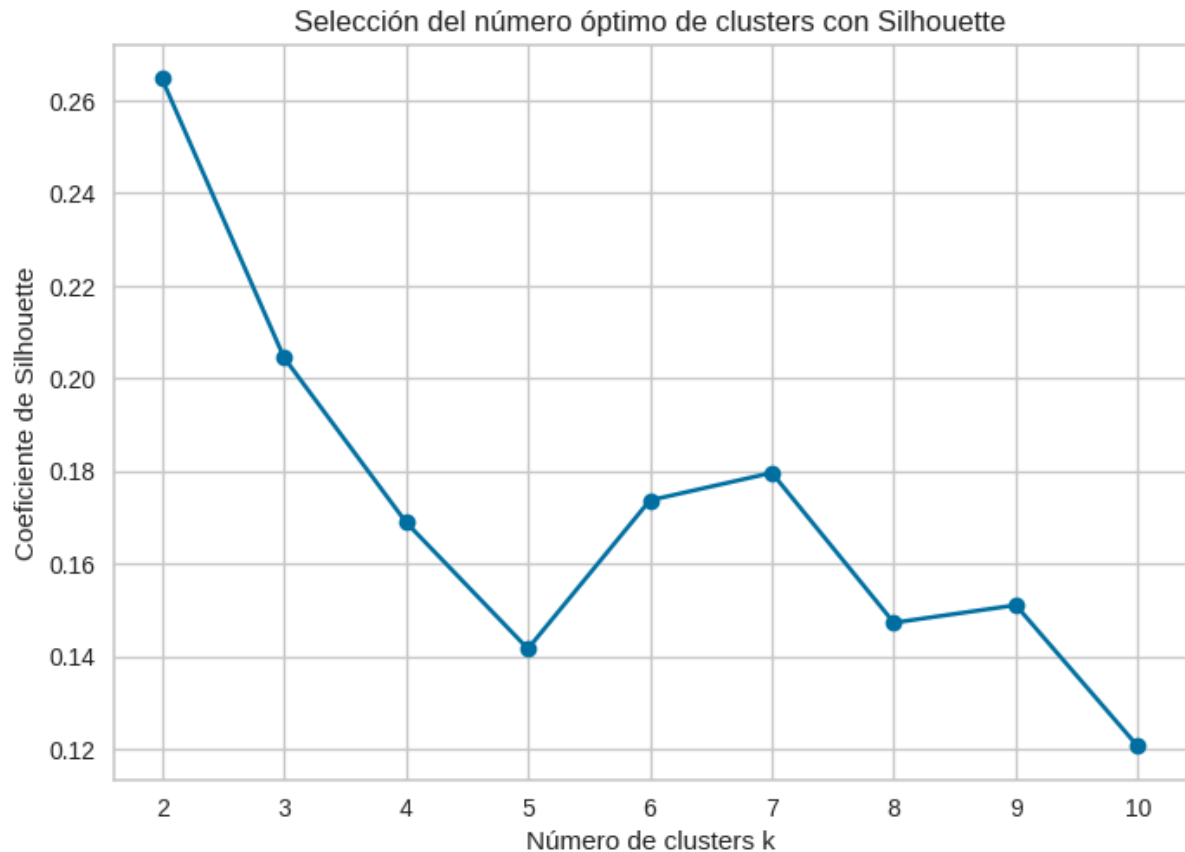
4. Estimación del número de grupos

Para determinar el K óptimo se combinaron dos métodos:

El método del codo muestra un quiebre claro en **K=3**, a partir del cual la reducción de inercia se vuelve progresivamente menor, sugiriendo que 3 es el número de clusters que mejor equilibra solidez y simplicidad.

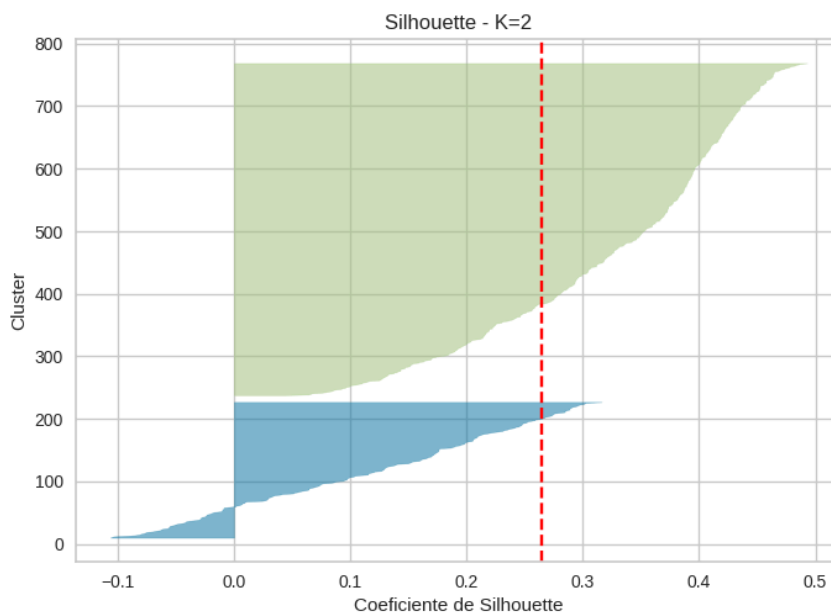


El índice de Silhouette por su parte indica **K=2** como óptimo (score ≈ 0.265), con **K=3** obteniendo un score muy cercano (≈ 0.205).

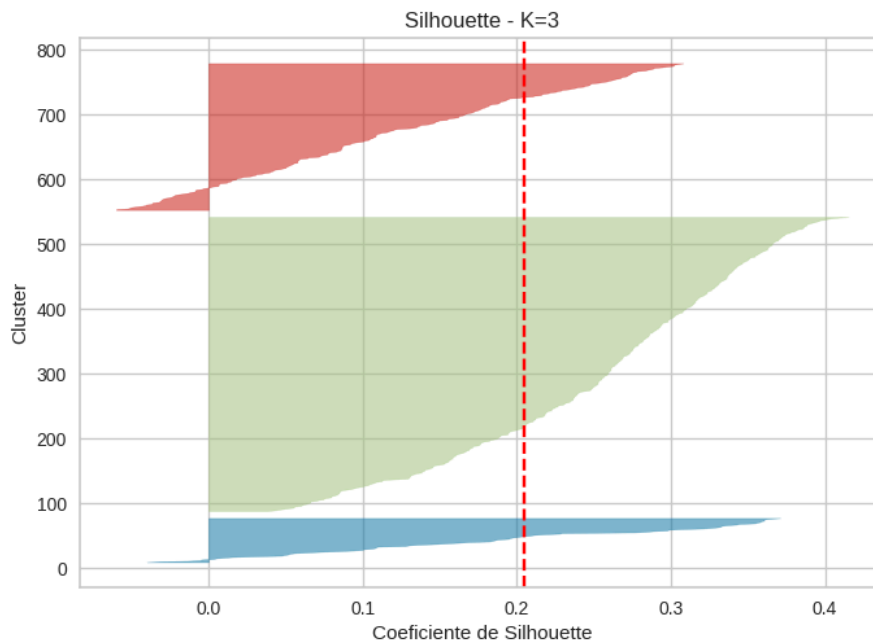


Dado que los métodos no coinciden, se evaluaron ambas opciones analizando el Silhouette plot individual de cada una.

Con **K=2** el Cluster 0 concentra 50 puntos negativos y un score promedio bajo (0.12), lo que evidencia que la partición es demasiado gruesa y fuerza canciones de perfiles distintos a convivir en el mismo grupo.



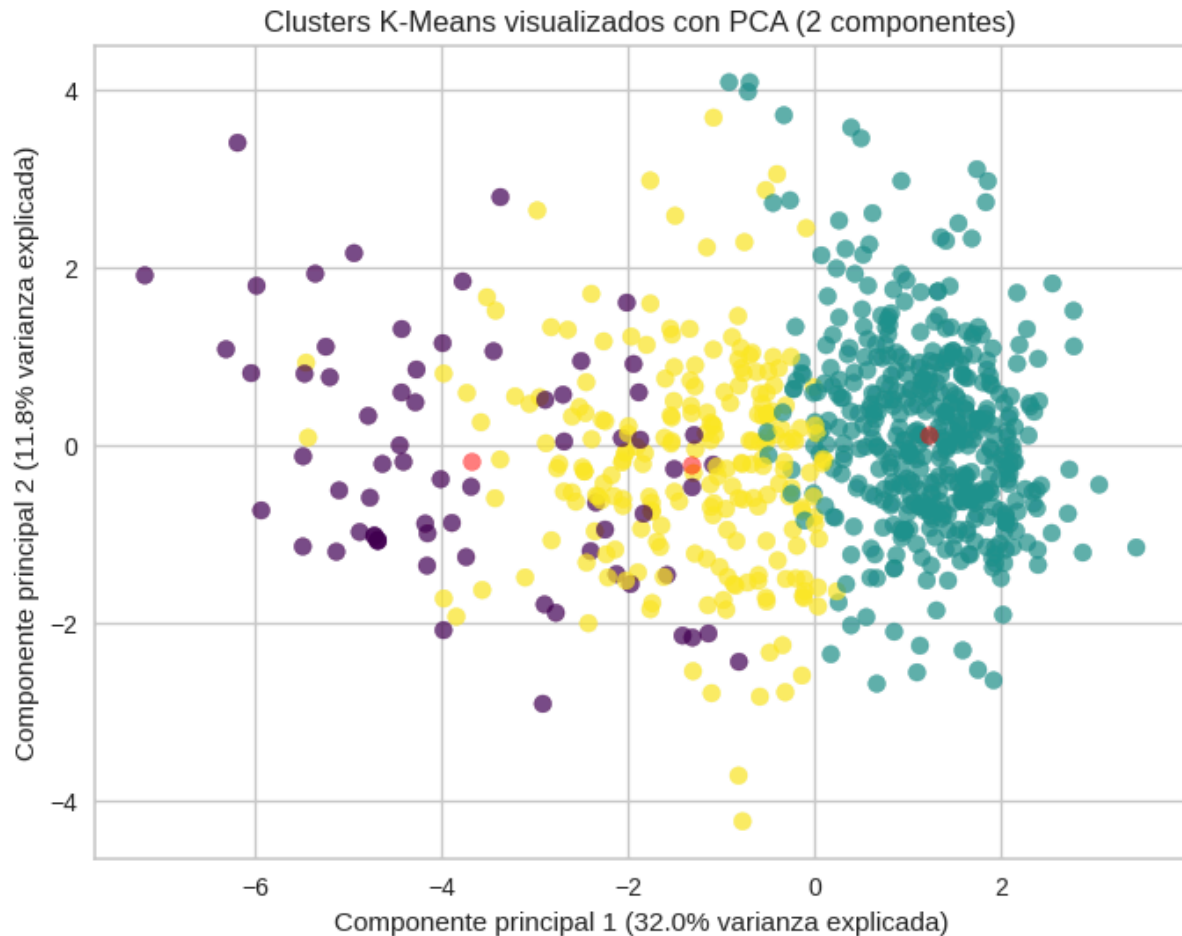
Con **K=3** la situación es más equilibrada: el Cluster 1 es muy cohesivo (454 canciones, 0 negativos, score 0.25), el Cluster 0 es pequeño pero compacto, y el Cluster 2 es el más heterogéneo.



Se optó por **K=3** como partición final ya que permite identificar un tercer perfil musical diferenciado a costa de un score global levemente menor.

5. Visualización de los grupos

Para visualizar los clusters se aplicó PCA, reduciendo las 11 features a 2 componentes principales con el único fin de poder graficar, ya que no es posible representar 11 dimensiones directamente. PC1 y PC2 explican en conjunto el 43.8% de la varianza, por lo que el gráfico es una aproximación: la superposición visible no implica que los clusters estén mal formados, ya que K-Means operó en el espacio original de 11 dimensiones. Aún así se puede apreciar que el cluster acústico/instrumental (violeta) se separa hacia la izquierda a lo largo de PC1, el cluster energético/bailable (verde) se concentra hacia la derecha, y el cluster moderado (amarillo) queda en una posición intermedia.



6. Descripción de los grupos

El análisis de los centroides revertidos a escala original permite identificar tres arquetipos musicales bien diferenciados:

Cluster 0 — Música Acústica e Instrumental (69 canciones)

Presenta **acousticness** muy alta (≈ 0.87), **instrumentalness** dominante (≈ 0.74), **energy** muy baja (≈ 0.22) y **loudness** bajo (-18.2 dB). Son canciones tranquilas, instrumentales y de mayor duración promedio, correspondientes a música clásica, ambient o folk instrumental. Es el cluster más pequeño pero el más compacto.

Cluster 1 — Música Energética y Bailable (454 canciones)

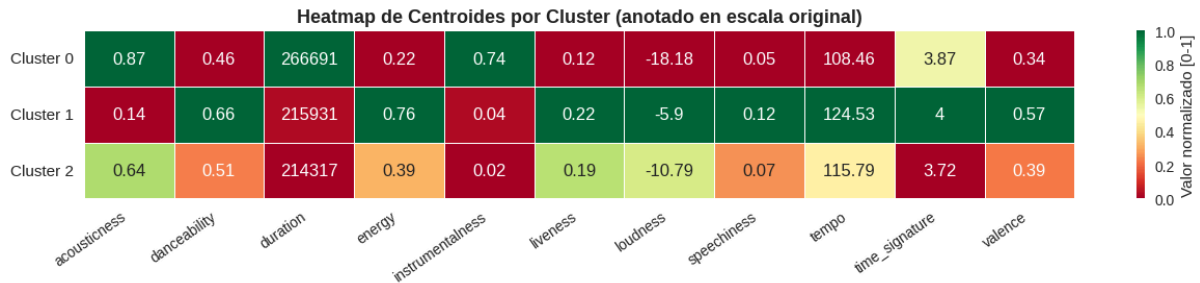
Presenta **energy** muy alta (0.76), **loudness** elevado (-5.9 dB), **danceability** alta (0.66) y **acousticness** muy baja (0.14). Son canciones producidas digitalmente, con ritmos fuertes y alto volumen. Es el cluster más grande y el más cohesivo, sin ningún punto negativo en el Silhouette.

Cluster 2 — Música Moderada y Balanceada (227 canciones)

Presenta valores intermedios en casi todas las variables: **acousticness** moderada (0.64), **energy** media (0.39), **loudness** moderado (-10.79 dB). Es el grupo más heterogéneo, lo

que se refleja en sus 35 puntos negativos en el Silhouette y corresponde a pop alternativo, rock suave y baladas.

El eje principal de separación entre los tres grupos es la combinación de energy, **acousticness** e **instrumentalness**, que captura el espectro desde música producida digitalmente hasta música acústica sin voz.



Tiempo dedicado

Integrante	Tarea	Promedio hs semanales
Bermudez, Agustin	EJ1 y EJ3 Armado de reporte	9 hs
Calderón, Tiago	EJ1 y EJ3 Armado de reporte	9 hs
Gonzalez Pautaso, Mateo	EJ1 y EJ3 Armado de reporte	9 hs
Moreyra, Santiago	EJ2 y EJ4 Armado de reporte	8 hs
Nieves, Maylen	EJ2 y EJ4 Armado de reporte	8 hs